Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABAJO DE FIN DE GRAO
GRADO EN INGENIERÍA INFORMÁTICA
MENCIÓN EN TECNOLOGÍAS DE LA INFORMACIÓN

Euro-Inf
Bachelor
awarded by
EQANIE

Desarrollo de un módulo de Autopsy para investigaciones forenses de imágenes asistido por Inteligencia Artificial

Estudante: Manuel Noya Vázquez

Dirección: Jose Vázquez Naya

A Coruña, septiembre de 2025.

A mi familia.

Agradecimientos

Deseo expresar mi más sincero agradecimiento a todas las personas que me han acompañado y apoyado durante la realización de este Trabajo de Fin de Grado. En primer lugar, a José, por su dedicación, su guía y por responder siempre con paciencia a mis dudas. También a Rubén, Martiño y a mis compañeros del CITIC, por su ayuda constante, por estar siempre dispuestos a colaborar y por contribuir, entre todos, a crear un entorno de trabajo motivador y agradable. A Mónica, gracias por ser un pilar esencial tanto durante este proceso como en mi día a día, por tu apoyo constante y por animarme incluso en los momentos más difíciles. A mi madre y a mi hermana, gracias por estar siempre ahí, por su comprensión y por facilitarme el espacio necesario para concentrarme en este trabajo. A mi padre, que ya no está, pero cuyo esfuerzo y aliento durante tantos años me impulsaron a seguir formándome. Su recuerdo y ejemplo siguen presentes en cada paso que doy. Y finalmente, a mi familia y amistades, por su apoyo incondicional y por estar cerca en todo momento. A todos, gracias por estar ahí y por formar parte de este proceso.

Resumen

La creciente cantidad de evidencias visuales en investigaciones forenses digitales evidencia la necesidad de herramientas para analizar imágenes de forma automatizada, precisa y escalable. La inteligencia artificial supone una tecnología clave para dotar a las herramientas forenses de este tipo de capacidades. Sin embargo, aunque existen soluciones comerciales que ya incorporan estas funcionalidades, su coste es elevado. Por otra parte, herramientas open source ampliamente utilizadas como Autopsy, todavía no disponen de estas prestaciones específicas.

Motivado por esta carencia, el presente proyecto propone el diseño e implementación de un módulo para Autopsy que integra modelos de visión por computador con el fin de asistir en el análisis forense de imágenes. La herramienta permite realizar detección de objetos, búsqueda por similitud visual y búsqueda por texto, todo ello sin necesidad de abandonar la plataforma, integrándola en el flujo de trabajo de análisis de evidencias. La arquitectura del proyecto sigue un enfoque cliente-servidor, donde el módulo de Autopsy se comunica con un backend externo, encargado de realizar la inferencia con los modelos de inteligencia artificial.

Los modelos de visión por computador disponibles en la herramienta son YOLO (v8 y v11), SSD, y Faster R-CNN, para la detección de objetos; y CLIP (ViT-B/32), utilizado tanto para búsquedas semánticas por texto como para búsqueda por similitud entre imágenes. El sistema permite al usuario configurar de forma sencilla, a través de la propia interfaz de Autopsy, el tipo de búsqueda, el modelo deseado y los umbrales de confianza, adaptando así el análisis a las necesidades específicas del caso.

Además del desarrollo, el trabajo incluye una evaluación exhaustiva, centrada en la integración del módulo en Autopsy y en su impacto en el análisis forense de imágenes. Se realizaron pruebas funcionales y una comparación con el análisis manual, constatando que el módulo mantiene la precisión en la detección de evidencias y reduce de manera significativa el tiempo y esfuerzo requeridos, en comparación con el análisis manual.

El proyecto se ha desarrollado siguiendo la metodología iterativo-incremental, lo que ha permitido garantizar la calidad mediante revisiones continuas y mejoras progresivas. Se definió una planificación inicial con seguimiento y ajustes cuando fue necesario, se mantuvieron reuniones periódicas con el director para validar hitos y se realizó un análisis detallado de costes y recursos.

Como resultado, se proporciona una solución abierta, accesible y extensible que puede servir de base para futuras investigaciones o despliegues en entornos forenses reales. El código y la documentación están disponibles públicamente como proyecto open source bajo la licencia GNU General Public License v3.0 (GPLv3).

Abstract

The growing volume of visual evidence in digital forensic investigations highlights the need for tools capable of analyzing images in an automated, accurate, and scalable way. Artificial intelligence provides a key technology to equip forensic tools with such capabilities. However, while commercial solutions with these functionalities exist, their cost is high. On the other hand, widely used open-source tools such as Autopsy still lack these specific features.

Motivated by this gap, this project proposes the design and implementation of a module for Autopsy that integrates computer vision models to assist in forensic image analysis. The tool enables object detection, visual similarity search, and text-based search, all without leaving the platform, seamlessly integrating into the evidence analysis workflow. The project architecture follows a client–server approach, where the Autopsy module communicates with an external backend responsible for model inference.

The computer vision models available in the tool include YOLO (v8 and v11), SSD, and Faster R-CNN for object detection, and CLIP (ViT-B/32), used for both semantic text search and image similarity search. Through Autopsy's own interface, users can easily configure the type of search, the desired model, and confidence thresholds, tailoring the analysis to the specific needs of each case.

In addition to the development itself, the work includes a comprehensive evaluation focused on the module's integration into Autopsy and its impact on forensic image analysis. Functional tests and a comparison with manual analysis confirmed that the module maintains detection accuracy while significantly reducing the time and effort required compared to manual methods.

The project was developed following an iterative-incremental methodology, ensuring quality through continuous reviews and progressive improvements. An initial plan with ongoing tracking and adjustments was defined, regular meetings with the supervisor were held to validate milestones, and a detailed cost and resource analysis was performed.

As a result, the project delivers an open, accessible, and extensible solution that can serve as a foundation for future research or deployment in real forensic environments. The code and documentation are publicly available as an open-source project under the GNU General Public License v3.0 (GPLv3).

Palabras clave:

- Informática forense
- Análisis de imágenes
- Visión por computador
- Búsqueda semántica
- Autopsy

Keywords:

- Digital forensics
- Image analysis
- Computer vision
- Semantic search
- Autopsy

Índice general

1 Introducción **1**

1.1 Contexto y motivación 1

1.2 Objetivos 4

1.3 Estructura del documento 4

2 Fundamentos teóricos **6**

2.1 Informática forense 6

2.2 Visión por computador 8

3 Estado de la cuestión **10**

3.1 Informática forense 10

3.1.1 Herramientas forenses tradicionales 10

3.1.2 Herramientas forenses con IA 13

3.2 Modelos de visión por computador 15

3.3 Limitaciones y oportunidades 17

4 Metodología y plan de proyecto **18**

4.1 Metodología 18

4.2 Planificación 20

4.3 Materiales y herramientas 22

4.4 Costes 23

5 Desarrollo **26**

5.1 Arquitectura del sistema 26

5.2 Incremento I: Módulo de Autopsy 27

5.2.1 Fundamentos del módulo 28

5.2.2 Interfaz de usuario 30

5.2.3 Registro e inicialización del módulo 32

5.2.4 Recuperación y procesamiento de imágenes 32
5.2.5 Preparación y envío de la solicitud al servidor IA 33
5.2.6 Procesamiento de la respuesta y creación de artefactos y etiquetas . . 35
5.2.7 Pruebas, logging y trazabilidad del módulo 40
5.3 Incremento II: Servidor de inferencia 41
5.3.1 Fundamentos del servidor 42
5.3.2 Carga y gestión de modelos 43
5.3.3 Manejo de peticiones y respuestas JSON 45
5.3.4 Manejo de imágenes y decodificación 46
5.3.5 Estrategias de inferencia por modo de ejecución 46
5.3.6 Procesamiento por lotes y gestión de memoria 47
5.3.7 Integración entre cliente y servidor 48
5.3.8 Pruebas, gestión de errores y robustez 49
5.4 Flujo de funcionamiento del sistema final 49

6 Evaluación **52**
6.1 Validación funcional 52
6.1.1 Pruebas de detección de objetos 52
6.1.2 Pruebas de búsqueda por similitud de texto 56
6.1.3 Pruebas de búsqueda por similitud de imagen 58
6.2 Evaluación del rendimiento 61
6.2.1 Metodología de evaluación 61
6.2.2 Evaluación de precisión 62
6.2.3 Evaluación de rendimiento 64
6.2.4 Comparación con análisis manual 65

7 Resultados **67**

8 Conclusiones y trabajo futuro **72**
8.1 Conclusiones 72
8.2 Trabajo futuro 73

A Manual de instalación y despliegue **76**
A.1 Instalación del módulo en Autopsy 76
A.2 Despliegue del servidor de IA 78

Bibliografía **80**

Índice de figuras

2.1 Proceso típico de análisis forense digital: identificación, adquisición, preservación, análisis y presentación . . . 7
2.2 Arquitectura típica de una red neuronal convolucional (CNN) . . . 9

3.1 Logo de Autopsy . . . 12

4.1 Esquema del modelo de desarrollo incremental . . . 19
4.2 Diagrama de Gantt del proyecto. Las barras en color morado muestran la planificación de cada tarea, mientras que las de color naranja reflejan las desviaciones sufridas con respecto a la estimación inicial. Los bloques azules corresponden a los hitos del proyecto . . . 21

5.1 Representación de los bloques lógicos principales de la arquitectura de la herramienta . . . 26
5.2 Interfaz de usuario para la configuración previa a la ejecución . . . 30
5.3 Interfaz de configuración de la funcionalidades de búsqueda por similitud: (a) búsqueda por texto y (b) búsqueda por imagen . . . 31
5.4 Formato y estructura de la petición HTTP POST enviada al servidor . . . 34
5.5 Organización de los artefactos dentro del panel de resultados de Autopsy . . . 36
5.6 Subgrupo de los artefactos visto desde el panel de resultados de Autopsy . . . 37
5.7 Organización de las categorías de etiquetas dentro del panel de resultados de Autopsy . . . 38
5.8 Artefacto correspondiente a una imagen con las clases detectadas, usadas para su etiquetado . . . 38
5.9 Artefactos de imágenes asociadas a una categoría concreta de etiquetas . . . 39
5.10 Estructura en disco de las imágenes procesadas, organizadas en carpetas por clases detectadas . . . 40
5.11 Estructura jerárquica del servidor de inferencia . . . 43

5.12 Salida de consola tras iniciar el servidor de inferencia 44
5.13 Formato y estructura de la respuesta HTTP POST enviada al módulo 45
5.14 Diagrama de secuencia del funcionamiento completo de la herramienta 50

6.1 Resultados de la ejecución de modelos de detección de objetos en Autopsy, con ejemplos de artefactos etiquetados en la interfaz. 54
6.2 Ejemplos de etiquetas generadas automáticamente por el sistema en la interfaz de Autopsy 55
6.3 Salida del servidor mostrando el envío de respuestas hacia Autopsy y el cálculo dinámico del tamaño de lote usado para cada modelo 56
6.4 Configuración seleccionada para la prueba de búsqueda por similitud de texto 56
6.5 Imágenes utilizadas en la prueba de búsqueda por similitud de texto 57
6.6 Presentación de los resultados de la prueba búsqueda de similitud de texto dentro de la interfaz de Autopsy 58
6.7 Etiquetas generadas automáticamente como resultado de la prueba búsqueda de similitud de texto dentro de la interfaz de Autopsy 58
6.8 Configuración seleccionada para la prueba de búsqueda por similitud de imagen 59
6.9 Conjunto de imágenes utilizadas en la prueba de búsqueda por similitud de imagen 59
6.10 Presentación de los resultados de similitud directamente en la interfaz de Autopsy 60
6.11 Etiquetas generadas automáticamente como resultado de la prueba búsqueda de similitud de imágenes dentro de la interfaz de Autopsy 61

Índice de tablas

3.1 Comparativa de modelos de detección y búsqueda empleados en el proyecto . 17

4.1 Duración estimada y real de las fases del proyecto 21
4.2 Coste asociados al personal, estimados y reales según los recursos del proyecto 24
4.3 Coste amortizado estimado y real de los recursos hardware empleados en el proyecto 25
4.4 Resumen de costes totales del proyecto 25

5.1 Tareas del incremento I (módulo de Autopsy) 27
5.2 Tareas del incremento II (servidor de inferencia) 41
5.3 Modos de ejecución del endpoint `/process` 47

6.1 Comparación de las detecciones realizadas por los modelos YOLOv8, YOLOv11, Faster R-CNN y SSD sobre cinco imágenes representativas 53
6.2 Resultados de la prueba de búsqueda por similitud de texto 57
6.3 Resultados de la prueba de búsqueda por similitud de imágenes 60
6.4 Resumen de métricas oficiales reportadas para los modelos utilizados en el proyecto 63
6.5 Tiempos de ejecución totales y promedios por imagen de los distintos modelos integrados en el sistema 65
6.6 Comparación del tiempo de análisis manual frente al uso de los distintos modelos integrados en el sistema 66

Capítulo 1

Introducción

Este capítulo presenta el contexto general del Trabajo de Fin de Grado (TFG) desarrollado. Se explica el problema actual que da pie a la motivación que condujo a desarrollar la solución propuesta, se mencionan objetivos concretos del proyecto y se indica cómo está estructurado el contenido de la memoria en sus diferentes capítulos.

1.1 Contexto y motivación

En la sociedad actual, la seguridad digital se ha convertido en un factor crítico. El número de ataques informáticos crece año tras año, afectando tanto a empresas como a instituciones públicas y a ciudadanos individuales. Informes de organismos como Europol o INCIBE alertan de un aumento constante en incidentes relacionados con cibercrimen, desde fraudes y robo de información hasta la distribución de material ilícito [1, 2]. Esta situación ha consolidado a la informática forense como disciplina esencial para esclarecer delitos, recuperar pruebas digitales y aportar evidencias con validez legal.

Cuando ocurre un incidente de seguridad o un ataque, la investigación forense digital se activa para reconstruir y esclarecer los hechos. El proceso de la informática forense se estructura en cinco fases [3]: (i) identificación, que consiste en la localización y reconocimiento de las fuentes potenciales de evidencia digital; (ii) adquisición, mediante la creación de copias forenses exactas que garanticen la integridad de los datos originales; (iii) preservación, encargada de mantener la cadena de custodia y proteger la evidencia frente a alteraciones; (iv) análisis, centrado en la interpretación técnica de la información extraída, la correlación de eventos y la detección de indicios relevantes; y (v) presentación, que implica la documentación exhaustiva y la comunicación clara de los hallazgos en un marco judicial o administrativo.

En este proceso, una figura clave es el perito forense digital [4, 5], profesional especializado en aplicar estas técnicas y en traducir los hallazgos técnicos a conclusiones que puedan ser comprendidas por jueces y abogados. La presión sobre este rol ha aumentado en los últimos

años debido al enorme volumen de información a analizar.

La cantidad de datos en una investigación típica es cada vez mayor. Casos que duran entre seis y dieciocho meses pueden implicar la revisión de más de 100.000 archivos digitales, muchos de ellos imágenes [6]. A ello se suma que en los últimos años el almacenamiento en la nube, los dispositivos móviles y las redes sociales han multiplicado el volumen y la diversidad de evidencias disponibles, lo que incrementa de manera significativa la carga de trabajo de los equipos forenses [7, 8].

Este crecimiento desmedido de información genera un cuello de botella en la fase de análisis. Cuanto más datos deben procesarse, más se prolonga el trabajo y más aumenta la posibilidad de pasar por alto pruebas relevantes. Informes de organismos especializados, como ENISA o el NCMEC, destacan que el gran volumen de evidencias digitales se ha convertido en uno de los mayores retos actuales de la informática forense, obligando a los laboratorios a destinar un tiempo desproporcionado a tareas repetitivas de clasificación y revisión manual [9].

Además, gran parte de estas evidencias son archivos multimedia, con un papel clave en delitos de gran impacto social, como la pornografía infantil, el tráfico de drogas y armas, el terrorismo, la radicalización o las extorsiones y amenazas digitales. En concreto, las imágenes constituyen una de las fuentes de evidencia más relevantes, ya que en muchos casos contienen pruebas directas de actividades ilícitas. La evidencia multimedia está presente en cerca del 90% de los procesos penales actuales [10], lo que sitúa al análisis visual como un pilar fundamental dentro de las investigaciones judiciales.

En todos estos escenarios, el perito debe visualizar manualmente miles de imágenes y vídeos para identificar contenido de interés. Este procedimiento consume una enorme cantidad de tiempo y recursos humanos, y en muchos casos los equipos forenses carecen de la capacidad suficiente para abordarlo con la rapidez que exigen los procesos judiciales.

Frente a esta problemática, la Inteligencia Artificial (IA) ofrece una vía de solución. En particular, la visión por computador ha demostrado ser capaz de procesar grandes volúmenes de datos visuales de forma automática y con altos niveles de precisión. Entre las tareas que estos sistemas pueden resolver se encuentran la detección de objetos y el etiquetado semántico, que han mostrado resultados prometedores en entornos prácticos [11].

Hoy en día ya existen modelos generales de visión por computador, disponibles públicamente como proyectos open source, que resuelven tareas como la detección de objetos, clasificación de imágenes o búsqueda por similitud, tanto visual como semántica por texto. Estas tareas pueden ser de gran valor en el contexto de un análisis forense, por ejemplo, de un disco duro o un dispositivo USB. Entre los modelos más representativos se encuentran You Only Look Once (YOLO) [12], Faster R-CNN [13], Single Shot MultiBox Detector (SSD) [14] o CLIP (ViT-B/32) [15]. Cada uno de ellos aporta un balance particular entre velocidad, exactitud y tipo de aplicación.

Los beneficios que estos modelos pueden aportar en entornos forenses digitales son:

- **Velocidad:** pueden procesar decenas o cientos de imágenes por segundo, frente a la inspección manual que consume horas o días.
- **Precisión y filtrado:** los modelos muestran un alto grado de exactitud en la detección de objetos, rostros o documentos. Aunque el análisis final dependa de la validación humana, pueden actúan como un primer filtro que permita al investigador centrarse solo en las imágenes sospechosas, o más relevantes para un caso concreto.
- **Búsqueda semántica:** permiten agrupar imágenes según su contenido o concepto compartido, facilitando al perito la identificación rápida de grupos de interés.
- **Consistencia y reproducibilidad:** los modelos de IA aplican criterios uniformes a cada imagen, evitando variaciones debidas a factores humanos como la fatiga o la subjetividad. Su funcionamiento sistemático facilita que el filtrado y la clasificación se realicen de manera homogénea, mejorando la trazabilidad y la reproducibilidad del análisis forense.
- **Detección de patrones ocultos:** la IA puede identificar patrones o anomalías sutiles en las imágenes que podrían pasar desapercibidos para el perito forense, aportando un nivel de exhaustividad difícil de alcanzar manualmente.

Actualmente, ya existen herramientas comerciales que incorporan estas capacidades de forma nativa, como Griffeye Analyze [16] o Magnet AXIOM [17]. Entre sus funcionalidades destacan la búsqueda semántica y la clasificación automática de grandes volúmenes de imágenes. Sin embargo, estas soluciones suelen tener un coste elevado, lo que limita su adopción en muchos laboratorios.

En contraste, las herramientas de código abierto más extendidas no cuentan aún con integración de modelos de IA. Una de las más destacadas es Autopsy [18], ampliamente utilizada en el análisis de discos y dispositivos de almacenamiento. Esta herramienta dispone de componentes para trabajar con imágenes, como galerías de visualización y extracción de metadatos EXIF [19], pero carece de módulos nativos que apliquen IA para clasificación o búsqueda avanzada de contenido visual.

Por todo ello, surge la necesidad de desarrollar un módulo específico para Autopsy que integre capacidades de visión por computador. Esta propuesta no solo busca mejorar la eficiencia en la gestión de grandes volúmenes de evidencias digitales, sino también aportar una solución abierta, extensible y accesible, que permita a los laboratorios forenses aprovechar el potencial de la IA sin depender de herramientas propietarias y costosas.

1.2 Objetivos

La problemática descrita en la sección anterior pone de manifiesto la necesidad de incorporar tecnologías de IA en herramientas de análisis forense digital. Autopsy, debido a su uso extendido y a su arquitectura modular, constituye la plataforma idónea para desarrollar un módulo que permita aplicar modelos de visión por computador en la búsqueda y clasificación de imágenes.

Este proyecto tiene como propósito dar respuesta a esa necesidad, diseñando e implementando un módulo que integre IA en el flujo de trabajo de Autopsy, de forma que los investigadores forenses puedan analizar grandes volúmenes de imágenes de manera más eficiente, rápida y precisa.

Objetivo principal:

El objetivo principal de este proyecto es el desarrollo de un módulo en Autopsy que integre tecnologías de IA para la búsqueda y análisis de imágenes en investigaciones forenses digitales.

Objetivos específicos:

Para poder alcanzar el objetivo principal, se plantean los siguientes objetivos específicos:

- **OBJ1:** Desarrollar un módulo que permita al investigador forense buscar y clasificar imágenes utilizando varios modelos de IA integrados en el flujo de trabajo de Autopsy.
- **OBJ2:** Integrar funcionalidades de detección y clasificación de objetos, búsqueda por similitud de imagen y búsqueda por similitud de texto, automatizando el etiquetado y la agrupación de imágenes según su contenido, facilitando así las investigaciones forenses.
- **OBJ3:** Evaluar la efectividad del módulo mediante casos de uso y métricas forenses (precisión, rendimiento y tiempo de respuesta), asegurando su fiabilidad y utilidad en escenarios reales de investigación.

1.3 Estructura del documento

Esta memoria está organizada en 8 capítulos que abarcan desde la presentación del problema y los objetivos, hasta la implementación concreta, la evaluación de resultados, conclusiones y líneas futuras de desarrollo e investigación:

- **Introducción:** En este capítulo se presenta el marco general del proyecto, detallando la relevancia del trabajo, los retos que pretende abordar y los objetivos principales que se desean alcanzar.

- **Fundamentos teóricos:** Ofrece una revisión de los conceptos y teorías fundamentales que sustentan el proyecto.
- **Estado de la cuestión:** Presenta un análisis de las soluciones y enfoques existentes, evaluando las herramientas y estudios previos relacionados para identificar las fortalezas, debilidades y vacíos que este trabajo pretende cubrir.
- **Metodología y plan de proyecto:** En esta sección se describe el enfoque metodológico empleado, detallando las fases del proyecto, así como el cronograma, los recursos y los costes estimados para llevarlo a cabo.
- **Desarrollo:** Representa el eje central del proyecto. En este apartado se detallan y fundamentan las tareas de implementación realizadas conforme a la metodología seleccionada. Además, se expone cómo se aplica la solución propuesta al problema planteado y se describe su estructura.
- **Evaluación:** Este capítulo tiene como propósito, por un lado, validar de forma funcional la herramienta en escenarios reales de investigación, y por otro, llevar a cabo una evaluación técnica que permita medir su efectividad, fiabilidad y rendimiento en función de los objetivos definidos.
- **Resultados:** En este capítulo se presentan los resultados obtenidos tras el desarrollo del proyecto. Se describen las funcionalidades finales de la herramienta, su comportamiento en pruebas de rendimiento y su utilidad en escenarios forenses reales. También se expone cómo se han cumplido los objetivos definidos inicialmente y se valoran los logros adicionales surgidos durante el proceso, así como el impacto potencial de la solución en el contexto de la informática forense.
- **Conclusiones y trabajo futuro:** Este último capítulo recoge las conclusiones obtenidas a partir del desarrollo del proyecto, y se mencionan las posibles futuras líneas de desarrollo o investigación para la continuación del mismo.

Capítulo 2

Fundamentos teóricos

Antes de abordar el desarrollo del sistema propuesto, es necesario revisar ciertos conceptos fundamentales que permiten contextualizar adecuadamente el proyecto. En particular, se presentan las bases de la informática forense, como disciplina central en la recuperación y análisis de evidencias digitales, y de la visión por computador, como tecnología clave para automatizar la interpretación de imágenes dentro de este ámbito.

2.1 Informática forense

La informática forense [20] es una rama de la ciencia forense que se centra en la obtención, análisis, preservación y presentación de evidencias digitales de forma que sean legalmente válidas. Su objetivo es esclarecer hechos relacionados con delitos informáticos o incidentes de seguridad mediante el análisis riguroso de dispositivos electrónicos, sistemas operativos, redes o ficheros.

Un ejemplo reciente de la relevancia de esta disciplina se dio en Australia en 2024, cuando la difusión de un vídeo del político David Speirs, relacionado con presunto consumo de drogas despertó dudas sobre su autenticidad. Para esclarecer el caso, el tribunal solicitó la intervención del perito en informática forense Dr. Matthew Sorell, quien analizó diversos aspectos técnicos del archivo digital. Su informe concluyó que no existían evidencias de manipulación, determinando que el material era auténtico y consistente con los datos originales. [21].

Una figura clave en este proceso es el perito forense digital [22], un profesional capacitado para aplicar técnicas específicas de análisis que garanticen la integridad de las pruebas, así como su interpretación técnica en el contexto de una investigación.

El proceso típico de análisis forense consta de 5 fases bien definidas [23] (Figura 2.1):

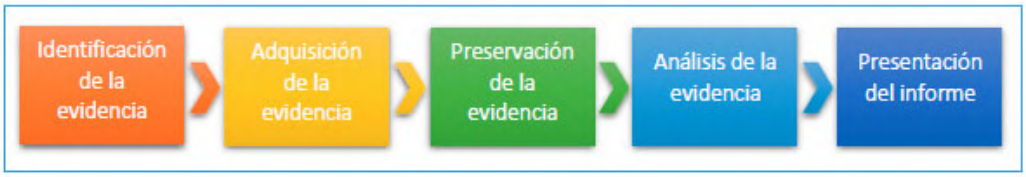


Figura 2.1: Proceso típico de análisis forense digital: identificación, adquisición, preservación, análisis y presentación

- **Identificación:** en esta fase se localizan las posibles fuentes de evidencia digital, como ordenadores, teléfonos móviles, discos externos o servicios en la nube. También se determinan los tipos de datos relevantes, por ejemplo registros de actividad, correos electrónicos o imágenes.

- **Adquisición:** consiste en obtener copias forenses exactas (bit a bit) de los dispositivos o archivos identificados, asegurando que el original no se modifica. Para ello se utilizan herramientas especializadas y bloqueadores de escritura. Un ejemplo sería clonar el disco duro de un ordenador incautado.

- **Preservación:** se centra en garantizar la integridad de las evidencias a lo largo del proceso. Incluye el uso de algoritmos hash para comprobar que no hay alteraciones y la documentación de la cadena de custodia. Por ejemplo, cada soporte se entrega acompañado de su valor hash y un acta de registro, como garantía de la cadena de custodia y protección frente a manipulaciones.

- **Análisis:** en esta fase se estudian las evidencias con detalle para extraer información útil. Esto puede incluir recuperar archivos eliminados, revisar registros del sistema o analizar metadatos de fotografías. El objetivo es reconstruir los hechos a partir de los datos digitales.

- **Presentación:** finalmente, se elaboran informes claros y comprensibles que recogen los métodos utilizados y los hallazgos obtenidos. Estos documentos se presentan en un proceso judicial y pueden incluir cronologías o gráficos que faciliten la interpretación de la evidencia.

Este proceso resulta fundamental para comprender los pilares sobre los que se ha construido la herramienta desarrollada, siempre orientada a su aplicación dentro del contexto forense descrito.

Un aspecto complementario de gran importancia es el principio del orden de volatilidad de la información, que establece que ciertos datos se pierden o alteran con mayor rapidez que otros, por lo que deben recogerse en primer lugar [24]. Por ejemplo, el contenido de la

memoria RAM o la tabla de conexiones activas de un servidor deben volcarse antes que, por ejemplo, sus discos duros, ya que estos datos desaparecerán una vez la máquina se apague.

En el caso de este trabajo, la propuesta se centra en el desarrollo de un módulo para Autopsy, una herramienta de código abierto ampliamente utilizada en el análisis forense de dispositivos de almacenamiento. Por ello, se sitúa de manera natural dentro de la fase de análisis del proceso forense (véase la Figura 2.1), concretamente, para el análisis de dispositivos de almacenamiento, como discos duros y memorias USB, entre otros.

2.2 Visión por computador

La visión por computador (del inglés, *computer vision*, CV) [25], es el campo de la IA que tiene como objetivo dotar a las máquinas de la capacidad de interpretar y comprender imágenes del mundo real. Basándose en principios de percepción visual, matemáticas aplicadas, procesamiento de señales y aprendizaje automático, combina técnicas de procesamiento digital de imágenes con modelos de aprendizaje profundo, permitiendo extraer patrones y características que a simple vista pueden pasar desapercibidos. De este modo, sistemas entrenados con grandes volúmenes de datos son capaces de reconocer rostros, identificar objetos o detectar anomalías en contextos muy diversos.

Esta tecnología es especialmente relevante en el contexto forense, ya que permite automatizar la revisión de grandes volúmenes de imágenes en busca de contenido sensible o ilícito. Esto reduce el tiempo necesario para las investigaciones y ayuda a los analistas a centrarse en los elementos más relevantes, aportando además un grado de exhaustividad y objetividad difícil de alcanzar exclusivamente con el trabajo humano.

Algunas aplicaciones típicas de la visión por computador incluyen:

- **Detección y reconocimiento de objetos:** identificar y localizar instancias de un objeto específico dentro de una imagen (por ejemplo, detectar coches o personas en una fotografía).
- **Clasificación de imágenes:** asignar una categoría general a la imagen completa (por ejemplo, clasificar si una imagen muestra un paisaje, un animal o un documento).
- **Análisis de escenas o comportamiento:** interpretar la relación entre múltiples elementos de la imagen o secuencia, como contar personas en una multitud o detectar comportamientos sospechosos en vídeo.
- **Búsqueda por similitud visual:** recuperar imágenes similares a una dada comparando características visuales (por ejemplo, buscar en una base de datos todas las imágenes parecidas a una fotografía de referencia).

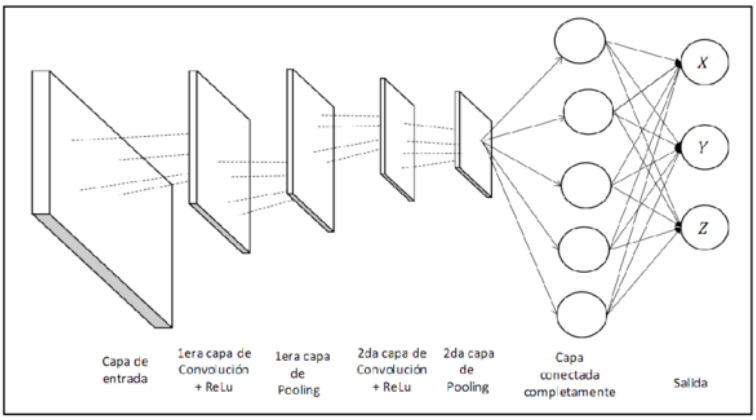


Figura 2.2: Arquitectura típica de una red neuronal convolucional (CNN)

- **Segmentación semántica:** dividir la imagen en regiones, asignando una etiqueta a cada píxel (por ejemplo, distinguir entre «carretera», «edificio» y «peatón» en una misma escena).

En términos de arquitectura, muchas soluciones actuales en visión por computador se basan en redes neuronales convolucionales (del inglés *Convolutional Neural Networks, CNN*) [26], ampliamente utilizadas para tareas de detección y clasificación de objetos. La Figura 2.2 muestra la arquitectura típica de una CNN.

También se emplean arquitecturas basadas en *Transformers* [27], especialmente útiles en contextos donde se requiere interpretar imágenes en conjunto con otros tipos de datos, como el texto. Estas arquitecturas han demostrado una gran capacidad para aprender representaciones visuales complejas y adaptarse a distintos dominios de aplicación.

La evolución reciente ha dado lugar a sistemas multimodales de gran escala, como Gemini (Google DeepMind) [28] o GPT-4V (OpenAI) [29], capaces de interpretar imágenes junto con texto y generar respuestas contextualizadas. Estas arquitecturas marcan el camino hacia una mayor integración entre distintas modalidades de información, incluyendo visión, lenguaje y, en algunos casos, audio o vídeo.

Cabe destacar que la explicación detallada de los modelos empleados en este trabajo se presenta en la Sección 3.2, donde se profundiza en sus características y rendimiento.

Capítulo 3

Estado de la cuestión

Una vez establecidos los conceptos clave que sustentan este trabajo, es necesario contextualizar la propuesta en relación con el estado actual del campo. Para ello, se analizan las líneas de investigación previas y las soluciones existentes que abordan el tratamiento automático de imágenes digitales dentro del entorno forense, así como los retos aún presentes que justifican el desarrollo de nuevas herramientas.

3.1 Informática forense

Dentro de este capítulo, resulta necesario revisar primero el ámbito de la informática forense, dado que constituye el marco principal en el que se inscribe este trabajo. En este apartado se describen las herramientas y enfoques más utilizados actualmente, con especial atención a aquellas soluciones aplicadas al análisis de imágenes digitales.

3.1.1 Herramientas forenses tradicionales

Comerciales

Existen soluciones comerciales consolidadas en el mercado forense que ofrecen flujos de trabajo "todo en uno", abarcando la adquisición, el procesamiento, el análisis y el reporte de evidencia digital. No obstante, estas herramientas no incorporan capacidades de IA para la clasificación o la búsqueda semántica de imágenes.

- **FTK (Forensic Toolkit) [30]**: proporciona indexación de texto completo, filtros avanzados por tipo de archivo y un motor de procesamiento en paralelo para acelerar la ingestión masiva de datos, así como galerías de vista previa de imágenes basadas en metadatos.

- **EnCase (OpenText) [31]**: combina adquisición forense, análisis de sistemas de archivos y generación de informes trazables, con potentes filtros de contenido estático, pero sin automatización basada en modelos de IA.
- **X-Ways Forensics [32]**: destaca por su rapidez en el escaneo de imágenes forenses y su flexibilidad con formatos exóticos, aunque su arquitectura propietaria impide integrar librerías de IA externas.
- **BlackLight (BlackBag Technologies) [33]**: orientado a macOS/iOS, ofrece vistas previas y análisis de artefactos de Apple, pero carece de módulos de visión por computador o clasificación automática de contenido.

Aunque estas herramientas son muy completas para la mayoría de tareas forenses, su enfoque basado en reglas y metadatos no escala bien al tratar volúmenes masivos de imágenes ni al detectar patrones complejos sin intervención manual.

De código abierto

Por otro lado, existen soluciones de software libre ampliamente utilizadas en el ámbito forense. Estas herramientas permiten realizar análisis digitales completos, desde la adquisición hasta el examen de evidencias, si bien suelen carecer de capacidades avanzadas de IA.

- **Kyno (by Lesspain Software) [34]**: aunque desarrollada inicialmente para entornos audiovisuales, Kyno ha sido empleada en contextos forenses por su capacidad de examinar y filtrar grandes volúmenes de archivos multimedia. Facilita el análisis preliminar de contenido visual mediante metadatos, filtros temporales y soporte para múltiples formatos, aunque no está orientada específicamente a la investigación digital.
- **CAINE (Computer Aided INvestigative Environment) [35]**: distribución GNU/Linux especializada que integra herramientas como Autopsy, Sleuth Kit, Volatility y más. Su ejecución en modo *live* evita alterar el sistema analizado, garantizando la preservación de las pruebas. CAINE destaca como una de las plataformas más completas y activas en el ámbito forense de código abierto.

The Sleuth Kit & Autopsy

Figura 3.1: Logo de Autopsy

Dentro del ecosistema de herramientas de código abierto, The Sleuth Kit (TSK) [36], una suite que mediante sus binarios y librerías permite analizar discos, sistemas de archivos y recuperar datos eliminados, y Autopsy [18], una aplicación con interfaz gráfica que permite gestionar de forma centralizada las funcionalidades de TSK y añadir módulos de análisis adicionales, ocupan un lugar central en la práctica forense digital. A diferencia de otras soluciones libres de carácter más generalista, estas plataformas han sido concebidas específicamente para el análisis de evidencias, consolidándose como estándares de facto en el ámbito académico, judicial y profesional. Su relevancia justifica un análisis en profundidad, no solo por la amplia adopción que han alcanzado, sino también porque constituyen la base tecnológica sobre la cual se desarrolla este proyecto.

Autopsy, en su versión 4.19 (con TSK 4.10.6 en Windows 10 y Ubuntu 20.04), distribuye una interfaz gráfica de código abierto que se apoya en TSK como motor de bajo nivel. Gracias a su arquitectura modular basada en *Ingest Modules*, Autopsy facilita la incorporación de nuevas funciones sin modificar su núcleo: cada módulo puede integrarse o eliminarse según las necesidades del caso forense. Aunque la herramienta no cuenta con integración directa de técnicas de IA para el análisis de imágenes, sí ofrece funcionalidades útiles que contribuyen al tratamiento y organización de contenido visual durante las investigaciones.

Entre sus módulos de ingestión nativos destacan el Image Gallery Ingest Module [37], que agrupa todas las imágenes extraídas en una galería visual; el Hash Lookup Module [38], que compara archivos contra bases de datos de hashes conocidas; y módulos para extraer metadatos EXIF [19] o realizar reconocimiento de texto (OCR) sobre imágenes [39]. Además de recuperación de archivos, análisis de sistemas de archivos y búsqueda de palabras clave, Autopsy permite el examen de correos, imágenes y vídeos, y la generación de informes detallados.

Asimismo, existen módulos , desarrollados por la comunidad y no integrados de forma nativa en Autopsy, como Tesseract OCR Ingest Module [40], que convierte contenidos gráficos en texto indexable, ampliando las capacidades de la herramienta. Sin embargo, tanto Autopsy como los módulos de la comunidad disponibles carecen de capacidades como la detección de objetos (”encontrar los objetos que aparecen en cada imagen”), la búsqueda semántica (“encontrar imágenes en base a una descripción textual: fotos con armas”) o la agrupación inteligente por similitud visual (”grupos de imágenes similares a una dada”).

3.1.2 Herramientas forenses con IA

A diferencia de las soluciones tradicionales, centradas en el análisis manual y en el uso de metadatos o reglas predefinidas, han comenzado a aparecer herramientas forenses que integran directamente técnicas de IA en sus flujos de trabajo. Estas plataformas marcan un punto de inflexión en la disciplina, ya que permiten automatizar tareas críticas como la clasificación de imágenes, la detección de patrones complejos o la búsqueda semántica en grandes volúmenes de datos visuales.

Suites forenses

Existen suites forenses que incluyen de serie capacidades de IA para el análisis de imágenes, ofreciendo un flujo de trabajo unificado sin necesidad de exportar datos:

- **Belkasoft Evidence Center [41]**: aunque orientado principalmente a análisis de disco y memoria, incorpora módulos de visión por computador para:
  - Detección automática de rostros, piel y texto en imágenes y vídeos, mediante redes neuronales integradas.
  - Clasificación básica de contenido multimedia (por ejemplo, vídeos, audio, fotos, etc.), incluyendo filtros para contenido explícito y detección de manipulaciones (plugin de forjado).

  Su precio de entrada, por una licencia perpetua, se sitúa en torno a los 2.800 €.

- **MobileEdit Forensic [42]**: plataforma forense profesional centrada en la extracción y análisis de evidencia digital en dispositivos móviles, que cuenta con módulos avanzados de IA especialmente relevantes para este proyecto:
  - Photo Recognizer: clasifica automáticamente imágenes y vídeos en categorías clave como armas, documentos, drogas, desnudez o monedas, ayudando a priorizar el material visual más relevante sin inspección manual.
  - Face Matcher: permite encontrar y comparar rostros en lotes masivos de imágenes o vídeos, facilitando la localización de personas específicas mediante reconocimiento facial.

  Adicionalmente, ofrece una amplia gama de funcionalidades útiles como adquisición lógica y física, recuperación de datos eliminados, análisis de aplicaciones y metadatos EXIF/GPS, extracción de datos en la nube y desbloqueo de dispositivos.

  El coste anual de una licencia de esta herramienta ronda los 2.500 €, lo que hace que sus funcionalidades resulten inaccesibles para entornos forenses con recursos limitados.

Durante el desarrollo del TFG, hemos tenido acceso a una licencia demo de la versión de MobileEdit Forensic PRO, facilitada por el equipo de MobileEdit. Esta experiencia fue clave para motivar la creación del módulo de IA en Autopsy: se pudo comprobar directamente cómo funcionalidades como Photo Recognizer y Face Matcher agilizan el análisis de grandes volúmenes visuales, así como su rendimiento y viabilidad, demostrando que integrar herramientas similares en Autopsy podía aportar capacidad avanzada sin depender de soluciones comerciales ni de elevados costes.

Estas herramientas demuestran que integrar modelos IA de forma nativa aporta un valor añadido en eficiencia y exhaustividad, pero a costa de licencias que pueden resultar elevadas para muchos laboratorios. El desarrollo de un módulo IA para Autopsy pretende ofrecer un compromiso: capacidades avanzadas con un rendimiento cercano pero a un coste mucho menor, aprovechando su arquitectura modular.

Proyectos y módulos alternativos

Actualmente muchas herramientas no integran IA para clasificación de imágenes de forma nativa. Para cubrir esta carencia, existen soluciones especializadas que aplican modelos de visión por computador y aprendizaje automático, aunque operan como herramientas o servicios independientes, haciendo necesario la división del flujo de trabajo:

- **PhotoDNA / Project VIC [43][44]**: utiliza técnicas de comparación de hashes para la detección automática de contenido explícito. PhotoDNA genera hashes robustos que identifican imágenes basadas en su estructura, permitiendo filtrar con eficiencia grandes volúmenes de archivos. Project VIC amplía este enfoque con conjuntos de hash colaborativos utilizados por múltiples agencias.

- **Griffeye Analyze [16]**: emplea IA para identificar rostros, objetos y patrones específicos en grandes repositorios de imágenes. Su motor de clasificación automática agrupa fotografías por similitud visual y permite búsquedas semánticas (por ejemplo, “mostrar todas las imágenes con armas”). Griffeye afirma una exhaustividad de hasta el 85% en detección de rostros en pruebas internas.

- **Magnet AXIOM [17]**: integra módulos de visión por computador que detectan y correlacionan datos visuales (por ejemplo, reconocimiento facial, identificación de documentos) con otras fuentes de evidencia digital. AXIOM proporciona una interfaz centrada en casos forenses, pero su flujo requiere exportar e importar visualizaciones desde otras herramientas.

- **CAPE (Computer-Aided Photo Examination) [45]**: plataforma de investigación académica que aplica CNN preentrenadas para clasificación de escenas forenses (interiores, exteriores, contenido violento).

Estas herramientas demuestran la viabilidad de aplicar IA al análisis de imágenes forenses, pero operan fuera de Autopsy, obligando a los peritos a usar flujos de trabajo fragmentados y a gestionar la transferencia de datos entre plataformas.

3.2 Modelos de visión por computador

En esta sección se describen los principales modelos de visión por computador empleados en el proyecto, presentando sus características, ventajas y limitaciones. El objetivo es proporcionar una base conceptual que permita comprender los criterios de selección y el papel que cada modelo desempeña en el análisis forense de imágenes.

Para este tipo de investigaciones resulta crucial elegir el modelo de visión por computador que mejor equilibre rendimiento y coste computacional según el caso de uso. En base a estos criterios, y al análisis exhaustivo de los modelos de visión por computador disponibles públicamente, se han elegido los siguientes para ser integrados en la solución desarrollada:

- **You Only Look Once (YOLO) V8/V11 [12]**: procesa cada imagen en una sola pasada, alcanzando velocidades de decenas de fotogramas por segundo en GPU. Es ideal para la detección rápida de objetos en grandes lotes de imágenes, a modo de primer filtrado (por ejemplo, para filtrar fotografías que contienen armas o documentos). La versión 11 representa una mejora significativa en precisión, aunque con un mayor tiempo de inferencia en comparación con la versión 8.

- **Faster R-CNN [13]**: utiliza una red de propuestas de región (RPN) seguida de una etapa de clasificación, lo que le confiere mayor precisión (*mAP* más alto) a costa de una velocidad menor. Es adecuado cuando la exactitud en la localización de objetos (por ejemplo, detección forense de detalles muy pequeños) es prioritaria.

- **Single Shot MultiBox Detector (SSD) [14]**: también opera en un único paso, usando múltiples escalas de detección que mejoran el rendimiento en objetos de distintos tamaños (por ejemplo, rostros pequeños en fotos de grupo). Mantiene velocidades comparables a YOLO, con una ligera penalización en los fotogramas por segundo (FPS).

- **Contrastive Language–Image Pretraining (CLIP) [15]**: modelo multimodal que proyecta imágenes y texto en un mismo espacio semántico. Permite búsquedas basadas en lenguaje natural (por ejemplo, "todas las fotos que parezcan pasaporte") sin necesidad de reentrenar el modelo, lo que resulta muy útil para consultas forenses *ad hoc*.

Durante el proceso de búsqueda inicial, se evaluaron otras alternativas, que finalmente quedaron descartadas por diferentes razones:

- **R-FCN [46]**: ofrece una detección casi tan precisa como Faster R-CNN, pero su velocidad de inferencia (aprox. ~170 ms por imagen) resulta mayor en la práctica.
- **RT-DETR (Detection Transformer) [47]**: introduce un enfoque basado en transformadores y ofrece buena precisión; sin embargo, su implementación estándar inicial mostraba tiempos de inferencia elevados.
- **Vision Transformers completos como ViT [48]**: a pesar de su precisión y versatilidad, su alto coste computacional y la necesidad de GPUs potentes incrementan notablemente el coste de uso. Aunque modelos como CLIP utilizan ViT como *backbone* visual, en este caso su aplicación se centra en la búsqueda semántica multimodal y no en la detección directa de objetos.

Los modelos seleccionados (YOLO, SSD, Faster R-CNN y CLIP) ofrecen un equilibrio adecuado entre velocidad, precisión y portabilidad, en consonancia con los objetivos planteados y las restricciones de hardware. En el caso de los detectores tradicionales (YOLO, SSD y Faster R-CNN), se trata de arquitecturas diseñadas para identificar y clasificar objetos en una imagen, asignando a cada región detectada una etiqueta perteneciente a un conjunto predefinido de clases.

El entrenamiento y la evaluación de este tipo de arquitecturas se realizan habitualmente sobre conjuntos de datos de referencia, entre los que destaca COCO (Common Objects in Context) [49], desarrollado por Microsoft. Este dataset incluye más de 200.000 imágenes etiquetadas manualmente con 80 categorías de objetos cotidianos capturados en contextos naturales y variados. La métrica *mean Average Precision* (mAP), indicada en la Tabla 3.1, es una medida común para evaluar la precisión de estos modelos sobre COCO.

Por otro lado, a diferencia de estos detectores basados en clases fijas, el modelo CLIP (Contrastive Language–Image Pretraining) ha sido entrenado para aprender la relación entre imágenes y descripciones en lenguaje natural. Esto le permite realizar tareas de recuperación semántica (como “mujer con cuchillo”) sin necesidad de una taxonomía cerrada, lo que lo convierte en una herramienta especialmente útil para búsquedas abiertas o ambiguas.

En la Tabla 3.1 se resumen las características principales de los modelos seleccionados.

Modelo	Velocidad GPU/CPU	Precisión aproximada	Ventaja	Uso
YOLOv8	∼100 FPS / 12 FPS	mAP ≈ 0.54 (COCO)	Máxima velocidad y arquitectura optimizada	Criba ultrarrápida de grandes lotes de imágenes
YOLOv11	∼90 FPS / 10 FPS	mAP ≈ 0.58 (COCO)	Mejor balance velocidad–precisión	Prefiltrado con mayor fiabilidad
SSD300	∼60 FPS / 6 FPS	mAP ≈ 0.44 (COCO)	Detección multiescala	Captura de objetos pequeños
Faster R-CNN	∼7 FPS / 1 FPS	mAP ≈ 0.60 (COCO)	Precisión máxima de localización	Análisis detallado de evidencias críticas
CLIP (ViT-B/32)	∼20 FPS / 2 FPS	Top-1 ≈ 0.36 (ImageNet)	Búsqueda semántica y similitud visual	Consultas en lenguaje natural y recuperación de imágenes similares

Tabla 3.1: Comparativa de modelos de detección y búsqueda empleados en el proyecto

3.3 Limitaciones y oportunidades

El análisis del estado de la cuestión ha permitido identificar varias limitaciones comunes. Las herramientas comerciales, aunque muy completas, presentan costes de licencia elevados que las hacen inaccesibles para muchos laboratorios. Las soluciones de código abierto, por su parte, ofrecen versatilidad y transparencia, pero carecen de módulos nativos de IA que permitan clasificar imágenes automáticamente, buscar contenido por similitud o realizar agrupaciones semánticas. Finalmente, los proyectos y servicios externos con IA demuestran la viabilidad de estas técnicas, aunque obligan a flujos de trabajo fragmentados basados en la exportación e importación de datos entre plataformas.

En este contexto, Autopsy representa una oportunidad especialmente relevante. Su arquitectura modular lo convierte en una base idónea para incorporar modelos de visión por computador sin necesidad de modificar su núcleo. Con YOLOv8 y YOLOv11 es posible realizar un prefiltrado ultrarrápido de fotografías con objetos de interés. Con SSD, gracias a su arquitectura multiescala, se pueden identificar objetos de diferentes tamaños en una sola pasada, desde rostros pequeños hasta vehículos en tomas panorámicas. Con Faster R-CNN se alcanza una localización de alta precisión en evidencias críticas. Finalmente, con CLIP se habilitan búsquedas en lenguaje natural y recuperación de imágenes similares directamente desde la interfaz de Autopsy.

La integración de estas capacidades no solo cubre las carencias funcionales actuales, sino que también permite un flujo de trabajo unificado dentro de Autopsy, reduciendo la dependencia de soluciones propietarias. Con ello, se facilita el acceso a técnicas avanzadas de visión por computador a un mayor número de laboratorios forenses, incluso en aquellos contextos con recursos limitados.

Capítulo 4

Metodología y plan de proyecto

En este apartado se expone la metodología iterativo-incremental seguida, así como el esquema de trabajo que estructura cada fase del proyecto. Además, se presenta una estimación global de tiempos ilustrada mediante un diagrama de Gantt y, finalmente, los materiales y costes asociados al proyecto.

4.1 Metodología

La metodología adoptada fue eminentemente iterativo-incremental [50], lo que permitió descomponer el sistema en entregas funcionales y acotadas que aportaron valor desde las primeras iteraciones. Cada ciclo recorrió análisis, diseño, implementación y pruebas, finalizando en un incremento verificable que se evaluó y validó antes de avanzar. Este enfoque facilitó la detección precoz de defectos, la retroalimentación continua y la adaptación a cambios, mitigando los riesgos asociados a una entrega monolítica al final del proyecto.

Como resultado de la aplicación de esta metodología, el trabajo se estructuró en dos grandes incrementos, correspondientes a los dos componentes principales del sistema:

- **Módulo de Autopsy:** Para integrar el análisis automatizado en el flujo forense, se desarrolló un módulo que permite configurar el sistema, extraer y enviar imágenes al servidor de IA, y posteriormente recibir los resultados con anotaciones y artefactos dentro de Autopsy.

- **Servidor de inferencia:** Con el fin de proporcionar la capacidad de procesamiento inteligente, se implementó un servidor encargado de recibir las imágenes enviadas desde el módulo, aplicar modelos de visión por computador y devolver respuestas estructuradas con las detecciones obtenidas.

Además, se complementó con la aplicación de prácticas de *Lean Software Development* [51], orientadas a optimizar el flujo de trabajo y minimizar desperdicios, lo que contribuyó a mejo-

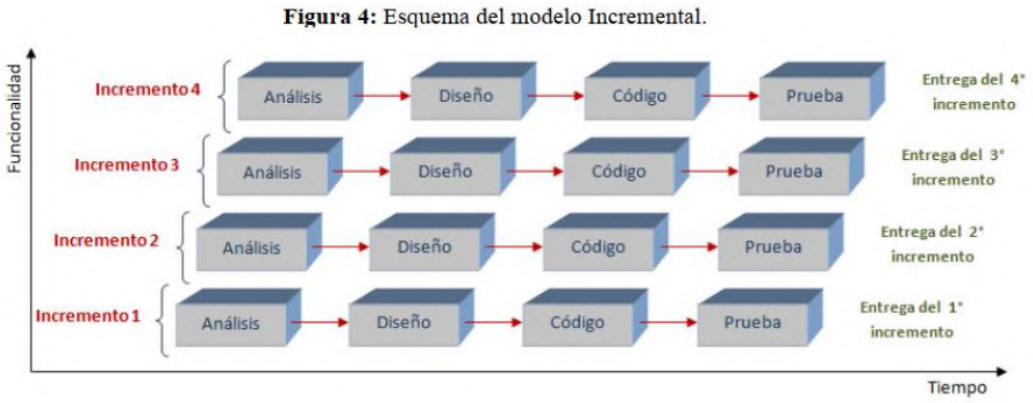


Figura 4.1: Esquema del modelo de desarrollo incremental

rar la eficiencia global del proyecto. Dichas prácticas se reflejaron en la eliminación continua de tareas innecesarias, como duplicidados en el envío de imágenes o procesamiento redundante. También permitieron la maximización del valor entregado en cada iteración, al tiempo que se mantenía el sistema en un estado funcional y "testeable".

A nivel organizativo, se utilizó un sistema Kanban [52] digital para gestionar las tareas, clasificándolas en columnas (Pendiente, En Progreso y Completado). Esta herramienta ofreció una visión clara del estado del proyecto y facilitó la priorización de funcionalidades según el avance real y los recursos disponibles. Como sistema de control de versiones para la gestión eficiente del código fuente se empleó Git [53], tanto para el módulo de Autopsy como para el servidor FastAPI [54], ambos alojados de forma conjunta en un único repositorio de GitHub. El uso de ramas para cada iteración, junto con una gestión sistemática de *commits* y revisiones, permitió mantener una trazabilidad clara de los cambios realizados y facilitó el trabajo incremental, así como la detección y corrección de errores en fases tempranas del desarrollo.

Asimismo, se mantuvieron reuniones de revisión con el director del proyecto, en las que se evaluaban los progresos alcanzados en cada ciclo y se replanificaban las siguientes tareas en función de los resultados obtenidos. Estas revisiones sirvieron como mecanismo de control de calidad, asegurando que el desarrollo evolucionara en línea con los objetivos técnicos y funcionales inicialmente definidos en la Sección 1.2.

Dado la relación directa del proyecto con los modelos de visión por computador, al comienzo del proyecto se planteó la necesidad de utilizar la metodología *Cross Industry Standard Process for Data Mining* (CRISP-DM) [6]. Esto surgía de la idea de que, quizás, durante la ejecución del proyecto, necesitaríamos entrenar o ajustar algún modelo para su especialización en tareas de casos forenses específicos (tráfico de drogas, armas, pornografía, etc.). Sin embargo, como se mostrará en la parte de evaluación (véase Capítulo 6), estos modelos ofrecen un rendimiento suficiente y estable en la gran mayoría de escenarios forenses, lo que permite

su uso transversal sin entrenar ni ajustar modelos *ad hoc* para cada tipología de caso. En consecuencia, un ciclo completo de minería de datos orientado al modelado (CRISP-DM) habría sido innecesario y no aportaría valor adicional frente a la estrategia de reutilizar modelos generales con alta capacidad de generalización. En todo caso, como se verá en el Capítulo 5, la arquitectura utilizada permite la integración de nuevos modelos en el futuro de forma sencilla, por si algún laboratorio o investigador forense quisiese emplear su propia solución.

4.2 Planificación

Para realizar la estimación inicial de la planificación del proyecto y de sus diferentes fases, se tomó como referencia la carga de trabajo asignada a la asignatura de Trabajo de Fin de Grado. Esta dedicación está establecida por el número de créditos *European Credit Transfer and Accumulation System* (ECTS) asignados a la materia, que en este caso son 12 ECTS, equivalentes a 300 horas de trabajo del estudiante [55]. Este esfuerzo estimado se distribuyó entre las fases del proyecto identificadas inicialmente, disponibles en la Tabla 4.1, de acuerdo con las indicaciones de la metodología y el conocimiento del equipo. La estimación de horas totales, teniendo en cuenta una jornada parcial de 4 horas diarias, supone un esfuerzo de 75 jornadas de trabajo.

A partir del reparto de horas entre las fases, se elaboró un diagrama de Gantt [56] (véase Figura 4.2). Esta herramienta no solo facilitó la visualización del calendario y de las dependencias entre tareas, sino que también permitió introducir márgenes de ajuste para absorber posibles desviaciones. A partir de esta planificación inicial se definió una línea base de trabajo, que sirvió como referencia durante la ejecución. Las variaciones con respecto a lo previsto fueron registradas y reprogramadas de manera controlada, procurando mantener en todo momento los hitos clave del proyecto.

Código	Nombre	Duración estimada (h)	Duración real (h)
F1	**Estudio inicial**	**90**	**105**
F1.1	Estudio del estado de la cuestión	35	40
F1.2	Estudio de la arquitectura de Autopsy	30	37
F1.3	Estudio de viabilidad y prototipo	25	28
F2	**Desarrollo**	**180**	**215**
F2.1	Incremento I: Módulo Autopsy	100	120
F2.2	Incremento II: Servidor inferencia	80	95
F3	**Documentación**	**30**	**50**
	TOTAL	**300**	**370**

Tabla 4.1: Duración estimada y real de las fases del proyecto

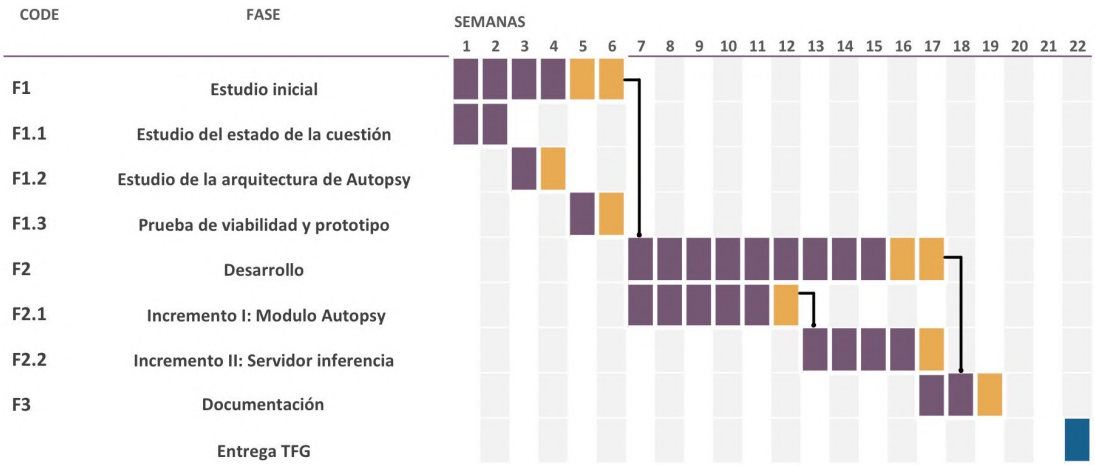


Figura 4.2: Diagrama de Gantt del proyecto. Las barras en color morado muestran la planificación de cada tarea, mientras que las de color naranja reflejan las desviaciones sufridas con respecto a la estimación inicial. Los bloques azules corresponden a los hitos del proyecto

4.3 Materiales y herramientas

Durante el desarrollo del proyecto se emplearon distintas herramientas y recursos tanto a nivel de hardware como de software, divididos entre un entorno local de desarrollo y una infraestructura externa de procesamiento.

Hardware

- **Portátil personal**
  - **Uso:** entorno de desarrollo, programación, pruebas iniciales y ejecución de herramientas de análisis forense.
  - **Características:**
    * Modelo: HP OMEN 15.
    * Sistema operativo: Windows 10 Home (versión 22H2).
    * Procesador: AMD Ryzen 7 4800H.
    * Memoria RAM: 16 GB.
    * Tarjeta gráfica: NVIDIA GeForce RTX 2060.
- **Máquina virtual del CPD**
  - **Uso:** entorno de despliegue de los modelos de IA, backend del módulo y ejecución de pruebas en entorno simulado.
  - **Características:**
    * Sistema operativo: Ubuntu 22.04 LTS.
    * Procesador: AMD EPYC 7763, 32 vCPU.
    * Memoria RAM: 125 GB.
    * Almacenamiento: 96 GB.

Software

- **Lenguajes y entornos de programación**
  - Python [57], como lenguaje principal para el desarrollo del servidor de IA.
  - Jython [58], empleado para integrar el módulo con Autopsy.
  - Visual Studio Code [59] como *Integrated Development Environment* (IDE), con extensiones específicas para Python, Jython y Git para control de versiones .
  - GitHub Codespaces [60], empleado de forma puntual como entorno de desarrollo remoto para realizar comprobaciones de código.

- **Frameworks y librerías de IA**
  - Ultralytics (YOLOv8 y YOLOv11) [61, 62] : empleado para la detección de objetos en imágenes dentro del conjunto de evidencias forenses.
  - Torchvision (Faster R-CNN) [63]: utilizado para tareas de detección y clasificación de objetos.
  - SSD (Single Shot MultiBox Detector) [64]: modelo rápido y preciso para detección de objetos en grandes volúmenes de datos.
  - CLIP (ViT-B/32 [65]: búsqueda semántica entre texto e imagen.
  - Pillow [66]: librería auxiliar para procesamiento y manipulación de imágenes.
  - psutil [67]: herramienta para monitorización de recursos del sistema.
- **Frameworks y herramientas backend**
  - FastAPI [54], framework web ligero de alto rendimiento en Python.
  - Docker [68], empleado para encapsular dependencias y facilitar la integración con Autopsy mediante contenedores reproducibles.
- **Pruebas e interoperabilidad**
  - Postman [69], utilizado para enviar peticiones simuladas al servidor y validar la interoperabilidad cliente-servidor.
- **Herramientas de acceso remoto**
  - Acceso a la *Virtual Private Network* (VPN) de la Universidade da Coruña (UDC), utilizada para acceder a la máquina virtual del *Centro de Procesamiento de Datos* (CPD) alojado en el Centro de Investigación en TIC (CITIC) y garantizar una conexión segura.

Debido a la complejidad técnica de algunos componentes y al ajuste de funcionalidades en base a las pruebas, el proyecto requirió finalmente 370 horas de trabajo, reflejando un incremento aproximado del 23,3% sobre lo previsto, siendo necesarias 93 jornadas.

4.4 Costes

Para la estimación del coste total del proyecto se ha considerado tanto el tiempo dedicado por el personal implicado como los recursos materiales necesarios para su desarrollo.

Para calcular el coste del personal se ha utilizado la herramienta de cálculo de contratos de investigación de la Universidade da Coruña (UDC) [70], que permite estimar el coste

Recurso	Coste (€/h)	Esfuerzo estimado (h)	Esfuerzo real (h)	Coste total estimado (€)	Coste total real (€)
Director de proyecto	100,00	10	15	1.000	1.500
Desarrollador	18,30	300	370	5.490	6.771
TOTAL		**310**	**385**	**6.490**	**8.271**

Tabla 4.2: Coste asociados al personal, estimados y reales según los recursos del proyecto

global de un contrato de investigación. Para el autor principal se tomó como referencia la categoría de "Técnico superior de apoyo a la investigación", por ser la que mejor se ajusta a las competencias técnicas y al nivel de dedicación requeridos en el presente proyecto. Según esta categoría, el salario bruto mensual mínimo, establecido por la UDC, es es de 1.462,50 €. Con esta especificación, y teniendo en cuenta el salario mensual, pagas extra, costes sociales y demás conceptos, el coste total del contrato asciende a 29.286,60 €.

Si se distribuye este coste sobre una jornada estándar de 1.600 horas anuales (equivalente a 40 horas semanales), el resultado es un coste por hora de 18,30 €, que se emplea como base para estimar el esfuerzo imputable al desarrollador. Por su parte, el coste del director del proyecto se valoró en 100 €/h, teniendo en cuenta su experiencia, nivel de especialización y el carácter técnico y metodológico de las tareas de supervisión.

En la Tabla 4.2 se presenta el coste asociado al personal implicado en el proyecto, mostrando tanto la estimación inicial como el esfuerzo finalmente realizado.

Además del coste del personal, se han considerado otros gastos asociados al uso de recursos físicos y servicios de infraestructura durante el desarrollo (véase Tabla 4.3):

- **Portátil personal:** Se empleó de forma continuada para tareas de desarrollo, pruebas e investigación. El equipo tiene un coste de adquisición aproximado de 1.000 €. Teniendo en cuenta el periodo de amortización típico de un equipo portátil, que son 48 meses, y calculando su amortización lineal con respecto a la duración del proyecto, el coste imputado al tiempo estimado inicialmente (4 meses, equivalentes a unas 300 horas) ascendía a 83,33 €. Sin embargo, debido a las desviaciones del proyecto, teniendo en cuenta las 370 horas reales finales (5 meses efectivos), el coste real amortizado ha sido de 104,17 €.

- **Máquina virtual en CPD:** En la fase final del proyecto se utilizó una máquina virtual alojada en un CPD, cedido por el CITIC [71]. Esta contaba con una configuración equivalente a la de un servidor profesional con 32 vCPUs y 125 GB de RAM. Según tarifas de mercado de proveedores de infraestructura en la nube [72], el coste estimado por hora para una máquina con estas características ronda los 1,20 €. Inicialmente, se estimó un uso de 25 horas, lo que supondría un coste de 30,00 €. No obstante, debido a las desviaciones del proyecto, el tiempo total ascendió a 32 horas alcanzando un coste real hasta 38,40 €.

Recurso	Precio base (€)	Tiempo de uso estimado (h)	Tiempo de uso real (h)	Precio amortizado estimado (€)	Precio amortizado real (€)
Portátil personal	1.000	300	370	83,33	104,17
Servidor	1,20€/h	25	32	30,00	38,40
TOTAL				**113,33**	**142,56**

Tabla 4.3: Coste amortizado estimado y real de los recursos hardware empleados en el proyecto

El coste total estimado del proyecto se situaba en 6.603,33 €, teniendo en cuenta tanto el tiempo previsto de dedicación del equipo como la amortización de los recursos materiales. No obstante, como consecuencia de desviaciones en la planificación y un uso superior al previsto de dichos recursos, el coste final alcanzó los 8.413,56 €, lo que representa un incremento del 27,41% respecto a la estimación inicial, tal y como se resume en la Tabla 4.4.

Recurso	Coste estimado (€)	Coste real (€)
Coste personal	6.490,00	8.271,00
Coste hardware	113,33	142,56
TOTAL	**6.603,33**	**8.413,56**

Tabla 4.4: Resumen de costes totales del proyecto

Capítulo 5

Desarrollo

El objetivo del desarrollo de esta herramienta es construir un sistema modular, eficiente y fácilmente integrable dentro del entorno de Autopsy, que permita al perito forense acelerar el análisis de imágenes digitales mediante técnicas de inteligencia artificial. Para ello, el sistema se ha concebido siguiendo un enfoque desacoplado, lo que facilita el mantenimiento independiente de cada componente, permite su futura ampliación (por ejemplo, con nuevos modelos o modos de búsqueda) y aprovecha herramientas de código abierto para garantizar un coste reducido y una alta compatibilidad con entornos forenses ya existentes.

El proyecto se estructura en dos incrementos principales: el módulo de procesamiento de imágenes para Autopsy y el servidor de inferencia basado en FastAPI [54] que aloja los modelos de visión por computador. En este capítulo se describe el desarrollo de dichos incrementos, que en conjunto conforman la herramienta forense propuesta.

5.1 Arquitectura del sistema

Para entender el sistema, su funcionamiento y cómo se comunican cada una de sus partes, en primer lugar se exponen los bloques lógicos principales de la arquitectura (Figura 5.1).

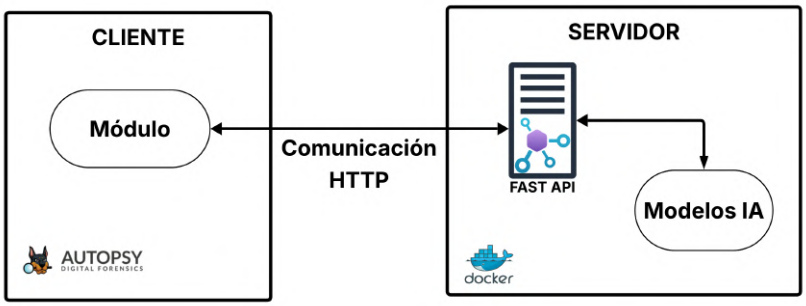


Figura 5.1: Representación de los bloques lógicos principales de la arquitectura de la herramienta

- **Autopsy:** herramienta forense base que carga el módulo desarrollado.
- **Módulo Python/Jython:** comprende la lógica principal, es detectado por la herramienta previamente mencionada y sirve como conexión con el servidor de inferencia.
- **Servidor IA (FastAPI + modelos):** carga los modelos, recibe las imágenes, realiza inferencias y devuelve anotaciones.
- **Canal de comunicación:** RESTful API vía HTTP [73], diseñada para ser ligera y robusta.

La herramienta se basa en una arquitectura cliente-servidor desacoplada [74], donde el cliente a su vez se ejecuta sobre la herramienta de Autopsy. De esta forma se facilita la escalabilidad, el mantenibilidad y la sustitución o mejora de bloques por separado.

5.2 Incremento I: Módulo de Autopsy

El módulo de Autopsy constituye el primer incremento del sistema, desarrollado siguiendo un ciclo iterativo compuesto por las fases de análisis, diseño, codificación y pruebas.

Durante la fase de análisis se identificaron las tareas necesarias para el desarrollo del módulo, recogidas en la Tabla 5.1. Estas tareas incluían desde la integración inicial en Autopsy hasta procesos como la comunicación con el servidor de inferencia, la recepción de resultados y su incorporación en forma de etiquetas, artefactos y anotaciones en el entorno forense.

Posteriormente, en la fase de diseño se definieron la arquitectura del módulo, su integración como *Datasource Ingest Module* [75], la interfaz gráfica de configuración y el flujo de interacción con el servidor de IA. En la tarea de codificación se implementaron las funcionalidades planificadas, y durante la fase de pruebas se verificó su correcto funcionamiento en

Tareas	Descripción
T1.1	Integración inicial del módulo en Autopsy.
T1.2	Implementación de la funcionalidad de extracción y copia de imágenes del sistema forense.
T1.3	Desarrollo del sistema de selección de modelo IA y ajuste de umbrales de confianza.
T1.4	Implementación del sistema de envío de imágenes al servidor externo mediante HTTP POST.
T1.5	Recepción y tratamiento de resultados: etiquetado, artefactado y anotaciones en Autopsy.

Tabla 5.1: Tareas del incremento I (módulo de Autopsy)

un entorno local, a la espera de su integración con el segundo incremento que se recoge en la Sección 5.3.

De esta forma, el ciclo completo culminó en un incremento funcional plenamente integrado en Autopsy, capaz de gestionar la configuración, extraer y enviar imágenes y presentar los resultados enriqueciendo el análisis forense de manera automatizada.

5.2.1 Fundamentos del módulo

La implementación de este módulo comenzó en la tarea T1.1 recogida en la Tabla 5.1, la cual estuvo centrada en su integración inicial en Autopsy.

Autopsy permite la extensión de sus capacidades mediante distintos tipos de módulos forenses, cada uno orientado a una fase concreta del análisis digital:

1. ***Ingest modules***: procesan automáticamente los datos de la evidencia mientras se incorporan al caso.
2. ***Report modules***: generan informes finales a partir de los resultados obtenidos.
3. ***Data source modules***: añaden nuevas fuentes de datos al caso.
4. ***Content viewer modules***: permiten visualizar y explorar de forma específica el contenido de los archivos.

En este proyecto, se ha implementado un *Datasource Ingest Module* [75], el cual se ejecuta durante la fase de ingestión del caso, tiene acceso a todos los archivos extraídos del dispositivo analizado y está diseñado para operar sobre cada fuente de datos incorporada al caso. Considerado el más apropiado cuando se desea procesar el conjunto completo de datos, especialmente en tareas como el análisis masivo de imágenes, ya que permite acceder directamente a todas las fuentes de datos incorporadas al caso y actuar sobre ellas sin intervención del usuario.

Desde el punto de vista del entorno de desarrollo, el módulo se ha programado en Jython, una implementación de Python 2.7 que permite su ejecución sobre la *Java Virtual Machine* (JVM) [76]. Esta elección viene impuesta por el propio entorno de Autopsy, que utiliza Jython como motor para ejecutar scripts. Sin embargo, esta decisión conlleva una limitación importante: Jython sólo es compatible con Python 2.7 y no soporta librerías modernas desarrolladas para Python 3, precisamente las empleadas por los modelos de IA que se pretenden integrar. En consecuencia, no es posible cargar dichos modelos directamente desde el módulo de Autopsy, lo que hace necesario establecer una comunicación con un entorno externo en el que puedan ejecutarse.

En este primer incremento esta restricción se resolvió de forma provisional mediante un script local simplificado que permitía validar el flujo del módulo. Esta solución sería poste-

```
# Factoría del módulo
class AIObjectDetectionFactory(DataSourceIngestModuleFactoryAdapter):

    def getModuleDisplayName(self):   # Nombre del módulo en Autopsy
    def getModuleDescription(self):   # Breve descripción del módulo
    def getModuleVersionNumber(self): # Versión del módulo
    def isDataSourceIngestModuleFactory(self): # Crea módulos DataSource
    def hasIngestJobSettingsPanel(self):      # Panel de configuración
    def getDefaultIngestJobSettings(self):    # Configuración por defecto
    def getIngestJobSettingsPanel(self):      # Panel gráfico de conf.
    def createDataSourceIngestModule(self, settings):
            # Instancia el módulo con settings

# Panel de configuración del módulo
class AIModelSettingsPanel(JPanel):

    def __init__(self):                       # Constructor
    def initComponents(self):                 # Construcción del panel
    def selectMainFunctionality(self, event): # Selección funcionalidad
    def selectSimilarityType(self, event):    # Búsqueda por similitud
    def chooseClipImage(self, event):         # Imagen de referencia
    def customizeComponents(self):            # Ajustes del panel
    def setAIModeFlag(self):                  # Activar/desactivar modelo
    def getSettings(self):                    # Configuración del usuario

# Clase principal del módulo
class AIObjectDetectionModule(DataSourceIngestModuleAdapter):

    def __init__(self, settings):               # Inicialización
    def startUp(self, context):                 # Tareas de inicio
    def process(self, dataSource, progressBar): # Procesamiento fuente
    def shutDown(self):                         # Liberar recursos

# Método interno para envío y artefactos
def _send_batch_and_create_artifacts(self):
        # Envío al servidor y generación en Autopsy
```

Listado 5.1: Clases y métodos principales del módulo

riormente reemplazada en el Incremento II (Sección 5.3) por un servidor de inferencia desacoplado, más robusto y escalable.

A partir de esta base, el desarrollo del módulo requirió implementar una serie de métodos fundamentales definidos por la arquitectura de Autopsy, que conforman el ciclo de vida del módulo durante su ejecución. A partir de estos métodos base hemos incorporado funciones auxiliares personalizadas, que serán llamadas desde los puntos clave del flujo principal, manteniendo la coherencia con el núcleo del sistema y permitiendo una mayor modularidad y control del proceso.

El Listado 5.1 presenta un esquema de la arquitectura base del módulo, que se desarrollará en las secciones siguientes.

5.2.2 Interfaz de usuario

Continuando con la tarea T1.1 recogida en la Tabla 5.1, la interfaz del módulo es clave para una experiencia de uso óptima. En nuestro caso, su diseño se centró en dos objetivos principales:

1. Facilitar una navegación intuitiva y clara.
2. Permitir la configuración completa del módulo.

Una vez instalado el módulo, este aparece entre la lista de Autopsy, permitiendo seleccionar y acceder a su configuración previa a la ejecución. En la Figura 5.2 se muestra el resultado final de la interfaz configurable.

En la Figura 5.2 se puede apreciar, a la izquierda, la lista de módulos disponibles donde se sitúa el desarrollado en este proyecto, y a la derecha, la configuración del mismo. La cual se divide según la funcionalidad principal, pudiendo elegir entre detección de objetos o búsqueda por similitud. En este caso, se ha seleccionado la detección de objetos, lo que habilita la posibilidad de escoger entre los distintos modelos de detección: YOLOv8, YOLOv11, Faster R-CNN o SSD.

La elección de detección de objetos y búsqueda por similitud como funcionalidades principales se tomó durante la fase de análisis, al definir los requisitos del sistema. Estas capacidades se consideraron las más útiles en un contexto forense, pues permiten localizar de forma automatizada evidencias visuales en grandes volúmenes de imágenes. La selección de los modelos de detección (YOLOv8, YOLOv11, Faster R-CNN y SSD) responde a los criterios de precisión

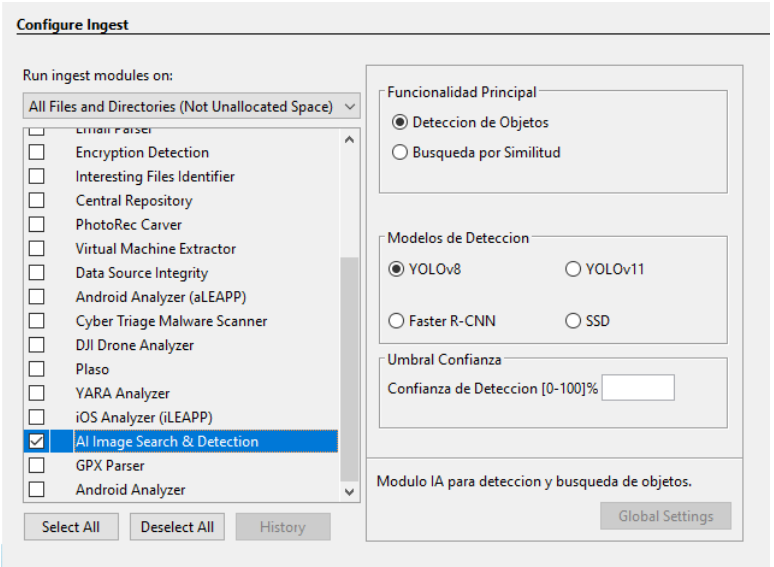
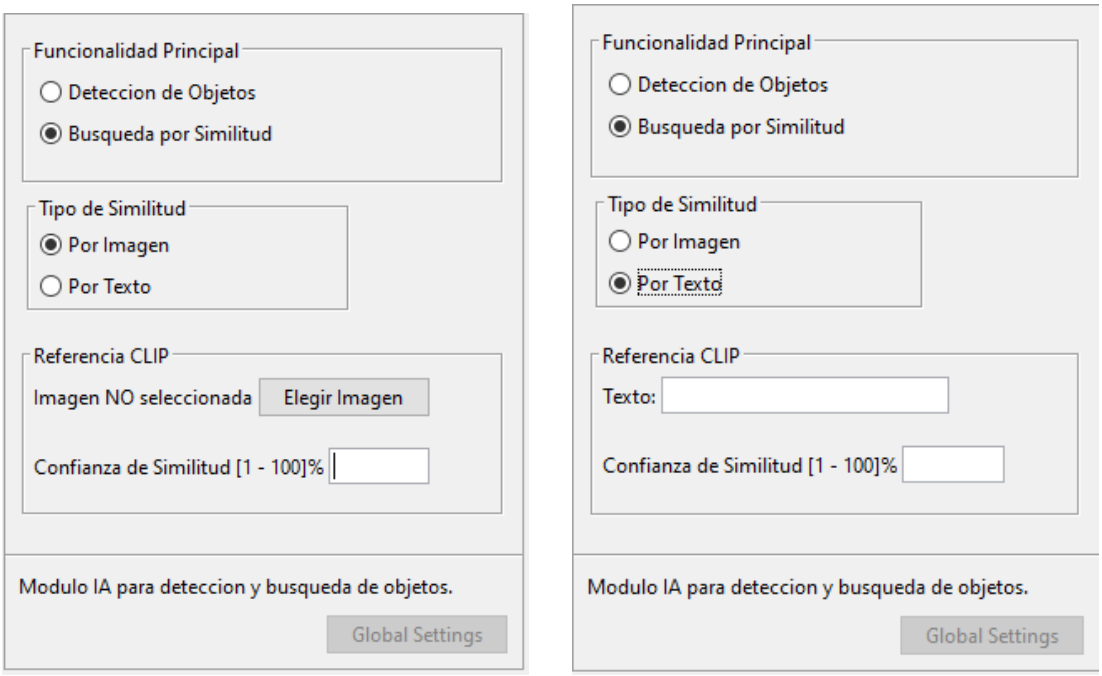


Figura 5.2: Interfaz de usuario para la configuración previa a la ejecución



(a) Búsqueda por texto (b) Búsqueda por imagen

Figura 5.3: Interfaz de configuración de la funcionalidades de búsqueda por similitud: (a) búsqueda por texto y (b) búsqueda por imagen

y eficiencia expuestos en la Sección 3.2, donde se analiza su idoneidad para tareas de investigación forense.

En las Figuras 5.3a y 5.3b se muestra la interfaz de configuración de la funcionalidad de búsqueda por similitud. Esta permite al usuario elegir el tipo de búsqueda, ya sea basada en texto o en imagen. En función de la opción seleccionada, el módulo solicitará una descripción textual (por ejemplo, "coche rojo") o bien una imagen de referencia.

En ambas funcionalidades (detección de objetos y búsqueda por similitud), se permite establecer un umbral de confianza, que determina el nivel mínimo de certeza que debe alcanzar una predicción para ser mostrada en los resultados. Filtrando así las detecciones menos fiables a gusto del usuario.

Los valores introducidos por el usuario durante la configuración del módulo se almacenan en la clase *AIModelSettingsPanel*, descrita en el Listado 5.1. Al comenzar la ejecución, estos parámetros se recuperan mediante el método *getSettings()*, que posteriormente se utiliza para adaptar el comportamiento del análisis a las opciones definidas. Además, se han implementado validaciones básicas en la propia interfaz, como la asignación de valores por defecto en caso de que no se proporcionen entradas válidas (por ejemplo, dejar el umbral vacío o introducir una ruta incorrecta).

Para implementar la interfaz gráfica del módulo, se han empleado principalmente las bibliotecas de Java Swing [77], integradas a través de Jython. Estas permiten construir componentes como *JPanel*, *JButton*, *JRadioButton* o *JFileChooser*, entre otros, organizados mediante gestores de disposición como *GridLayout* o *BoxLayout*, con el fin de garantizar una presentación clara e intuitiva de las opciones de configuración del módulo. Esta elección facilita la integración fluida con la interfaz nativa de Autopsy, proporcionando una experiencia de usuario coherente.

5.2.3 Registro e inicialización del módulo

Las operaciones que se detallan a continuación se corresponden con la tarea T1.3, representada en la Tabla 5.1, dedicada a la carga de configuraciones y al sistema de selección del modelo de IA.

Una vez configurado el módulo por el usuario y al presionar el botón *START*, Autopsy invoca automáticamente los métodos *__init__()* y *startUp()*, descritos en el Listado 5.1, dando comienzo al proceso de ingestión de la fuente de datos y marcando el inicio del procesamiento real.

La clase *AIObjectDetectionModule*, se inicializa mediante el método *__init__()*, donde se cargan las configuraciones definidas por el usuario y se preparan las estructuras internas necesarias, como la lista de imágenes a procesar o el contador de archivos. También se define una función de registro personalizada (mediante logs) que facilitará la trazabilidad del módulo a lo largo de su ejecución, y cuya implementación se detalla en 5.2.7.

Posteriormente, Autopsy ejecuta el método *startUp()*, donde se recuperan las preferencias seleccionadas en la interfaz de usuario: modelo de IA, tipo de funcionalidad (detección o búsqueda), umbral de confianza, entre otros. En esta fase también se asignan valores por defecto en caso de errores o entradas vacías, se establece la dirección del servidor y se crea un directorio temporal dentro del caso activo para almacenar imágenes procesadas. Todos estos pasos quedan reflejados en los logs como mecanismo adicional de trazabilidad.

Con la configuración inicial completada y los parámetros del usuario correctamente cargados, el módulo está listo para comenzar el análisis sobre los datos del caso. Este proceso tiene lugar en el método *process()*, donde se recorren los archivos de la fuente de datos en busca de imágenes y se prepara la información que será procesada.

5.2.4 Recuperación y procesamiento de imágenes

La presente sección aborda la implementación de la tarea T1.2, recogida en la Tabla 5.1, dedicada a la extracción y copia de imágenes desde el sistema forense.

Una vez configurado el entorno y establecida la conexión con el servidor de inferencia, el módulo inicia la recuperación de archivos relevantes del dispositivo de almacenamiento

analizado.

En este proyecto, el objetivo principal es localizar y preparar imágenes que puedan contener información visual de valor forense. Para ello, el módulo accede directamente a la base de datos interna de Autopsy mediante una consulta SQL[78] sobre la Tabla *tsk_files* [79], lo que permite recuperar eficientemente todos los archivos asociados a la fuente de datos actual. A través de esta consulta se seleccionan las imágenes contenidas en el dispositivo de almacenamiento, filtrando también posibles directorios o entradas no relevantes.

Cada una de las imágenes localizadas es extraída y copiada a un directorio temporal específico, asociado al caso actual. Esta decisión responde tanto a motivos prácticos (preparar los archivos para su envío al servidor) como a criterios forenses, ya que garantiza que el archivo original permanece inalterado, manteniendo así la integridad de la evidencia.

Esta carpeta se sitúa en el directorio temporal del caso:

`C:\Users\<username>\AppData\Local\Temp\Autopsy\<casename>\Temp`

Además, durante este proceso se construye una estructura de datos interna que relaciona cada imagen extraída con su identificador dentro de Autopsy. Este vínculo resulta esencial para que los resultados obtenidos en tareas posteriores puedan ser correctamente asociados a los archivos originales del caso.

5.2.5 Preparación y envío de la solicitud al servidor IA

Una vez recopiladas y filtradas las imágenes válidas desde la fuente de datos, el módulo debe preparar la información necesaria para comunicarse con el servidor de inferencia y realizar el análisis con los modelos de IA. Este apartado cubre la tarea T1.4, recogida en la Tabla 5.1.

Construcción del lote de imágenes:

Durante el procesamiento, las imágenes son agrupadas en un lote único que será enviado en una sola petición HTTP. Este enfoque reduce la sobrecarga de red y mejora la eficiencia del sistema. Cada imagen se convierte a texto utilizando la codificación Base64 [80], que permite representar imágenes como texto, facilitando su envío por HTTP dentro de un JSON [81]. Se eligió este formato por su simplicidad y compatibilidad, evitando manejar archivos por separado. Aunque incrementa el tamaño, es ideal para transmisiones rápidas y directas al servidor. Junto con las imágenes, se agregan al lote los parámetros definidos por el usuario en la configuración del módulo, como el modelo de IA seleccionado, el tipo de análisis (detección de objetos, similitud semántica, etc.), el umbral de confianza o la referencia textual o visual, en caso de aplicarse.

En la Figura 5.4 se representa el formato y estructura de la petición HTTP POST enviada al servidor de inferencia .

Petición HTTP

Endpoint: /process

Método: POST

JSON Payload:

```
{
  "functionality": "objects",
  "model_flag": "Yv11",
  "threshold": 0.5,
  "images": [
    {
      "name": "imagen_001.jpg",
      "data": "base64_encoded_string_here"
    },
    {
      "name": "imagen_002.jpg",
      "data": "base64_encoded_string_here"
    }
  ],
  "query": "weapon"
}
```

Figura 5.4: Formato y estructura de la petición HTTP POST enviada al servidor

Este formato facilita que el servidor identifique el tipo de análisis requerido y reciba las imágenes de forma interpretable y estandarizada.

Envío mediante HTTP POST:

La comunicación se realiza mediante una petición HTTP POST hacia el endpoint `/process` del servidor. Este diseño RESTful garantiza compatibilidad, escalabilidad y facilidad de despliegue en redes locales o remotas. La dirección del servidor es configurable desde el propio módulo, lo que permitió abordar el desarrollo en dos incrementos diferenciados. En el primero, la funcionalidad del servidor se simuló mediante un script local en Python que cargaba los modelos y ejecutaba la lógica básica de inferencia, actuando como un sustituto simplificado. Este enfoque permitió validar el flujo completo del módulo en local y detectar posibles problemas sin necesidad de desplegar un servicio independiente. En el segundo incremento, este script fue reemplazado por el servidor real de inferencia, desacoplado del módulo e implementado como microservicio REST, lo que resolvió las limitaciones detectadas en la fase inicial y garantizó escalabilidad y mantenimiento a largo plazo.

La llamada HTTP se realiza dentro de una sección controlada del código, con gestión de errores incorporada: si el servidor no está disponible, si se produce un fallo de conexión o si la respuesta no cumple el formato esperado, se registran los incidentes para su posterior análisis. Todo este comportamiento fue comprobado en la fase de pruebas posterior a la codificación, verificando que los mecanismos de control respondían correctamente en diferentes escenarios.

Gestión de errores controlados:

Durante esta fase se aplican controles básicos para asegurar la robustez del sistema:

- Verificación de conectividad con el servidor.
- Validación del formato del JSON recibido como respuesta.
- Registro de mensajes de error en los logs.
- Posibilidad de cancelar el proceso si el usuario así lo indica.

5.2.6 Procesamiento de la respuesta y creación de artefactos y etiquetas

Una vez completada la inferencia por parte del servidor de IA (véase la Sección 5.3.3), se retorna una respuesta estructurada en formato JSON que contiene los resultados asociados a cada imagen enviada. Esta respuesta es procesada dentro del módulo para extraer la información relevante y generar artefactos en Autopsy que puedan ser analizados posteriormente por el investigador. Con ello se completa la tarea E1.5, recogida en la Tabla 5.1.

La estructura de la respuesta incluye, para cada imagen, una lista de objetos detectados o coincidencias semánticas, cada uno con atributos como el nombre del objeto, la confianza del modelo y, en algunos casos, metadatos adicionales como el texto de consulta o las coordenadas del recorte. Aunque la estructura concreta del JSON de respuesta se explicará más adelante (véase Figura 5.13), aquí basta con señalar que su diseño permite vincular de forma directa los resultados a cada archivo procesado.

Artefactos:

A partir de esta información recibida tras la respuesta, el módulo recorre los resultados y genera un artefacto forense por cada imagen que contenga al menos una predicción relevante. En Autopsy, un artefacto *(BlackboardArtifact)* es una unidad estructurada de evidencia que puede estar compuesta por distintos atributos *(BlackboardAttributes)*, como por ejemplo:

1. El nombre del objeto detectado.
2. La puntuación o nivel de confianza del modelo.
3. El texto de búsqueda introducido por el usuario (en modo CLIP).
4. La ruta temporal del recorte (si se ha guardado).
5. Identificadores para facilitar la trazabilidad.

Estos artefactos se asocian directamente al archivo original del caso a través del objeto *AbstractFile*, lo que garantiza que el resultado quede vinculado al elemento real del sistema investigado sin alterarlo. Este mecanismo resulta fundamental para preservar la integridad de la evidencia digital y asegurar la validez del análisis en un contexto forense.

Además, los artefactos generados por cada ejecución del módulo se agrupan automáticamente bajo la categoría *Interesting Files* dentro del panel de resultados de Autopsy. Cada grupo se identifica con una etiqueta descriptiva que combina el modelo utilizado y el umbral de confianza definido por el usuario, lo que permite organizar visualmente los resultados según los parámetros de inferencia.

Por ejemplo, si se ejecuta el modelo YOLOv11 con un umbral del 20% y posteriormente con uno del 85%, ambos conjuntos aparecerán como entradas diferenciadas dentro del árbol de resultados. Esta estructura jerárquica facilita la comparación entre ejecuciones y permite al investigador identificar rápidamente qué imágenes han sido marcadas en cada configuración.

La Figura 5.5 muestra esta organización dentro de la interfaz de Autopsy.

Si el usuario selecciona una de las salidas obtenidas en la Figura 5.5, por ejemplo "Detected With FRCNN Confianza 50pct (168)", encontrará expuestaslas 168 imágenes analizadas con sus respectivas anotaciones, como se aprecia en la Figura 5.6 . En este caso, se ha ejecutado el módulo sobre un conjunto de imágenes que contienen vehículos. Se puede observar cómo cada imagen ha sido anotada con un comentario personalizado.

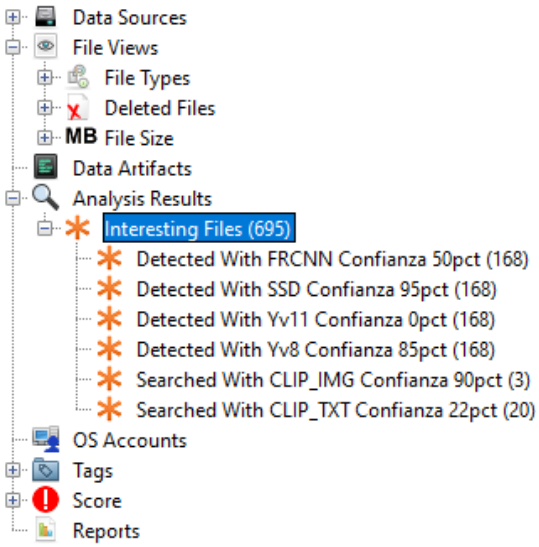


Figura 5.5: Organización de los artefactos dentro del panel de resultados de Autopsy

Source Name	S	C	O	Source Type	Comment	File Path
coches_13.jpg				File	Detections: car(0.832)	/LogicalFileSet1/lote_imagenes/coches_13.jpg
coches_14.jpg				File	Detections: car(0.997)	/LogicalFileSet1/lote_imagenes/coches_14.jpg
coches_15.jpg				File	Detections: car(0.999)	/LogicalFileSet1/lote_imagenes/coches_15.jpg
coches_16.jpg				File	Detections: car(0.946)	/LogicalFileSet1/lote_imagenes/coches_16.jpg
coches_17.jpg				File	Detections: car(0.535)	/LogicalFileSet1/lote_imagenes/coches_17.jpg
coches_18.jpg				File	Detections: car(0.999)	/LogicalFileSet1/lote_imagenes/coches_18.jpg
coches_19.jpg				File	Detections: car(0.981)	/LogicalFileSet1/lote_imagenes/coches_19.jpg
coches_2.jpg				File	Imposible Clasificar usando FRCNN con umbral 50%	/LogicalFileSet1/lote_imagenes/coches_2.jpg
coches_20.jpg				File	Detections: car(0.527)	/LogicalFileSet1/lote_imagenes/coches_20.jpg
coches_21.jpg				File	Detections: car(0.857)	/LogicalFileSet1/lote_imagenes/coches_21.jpg

Figura 5.6: Subgrupo de los artefactos visto desde el panel de resultados de Autopsy

En la columna de comentarios, se indica el objeto detectado (por ejemplo, 'car') junto con su nivel de confianza. También pueden aparecer mensajes como "Imposible clasificar usando FRCNN con umbral 50 %" cuando el modelo no ha podido hacer una predicción que supere el umbral establecido. Además, la columna "File Path" enlaza directamente con la ruta del archivo dentro del sistema de Autopsy, permitiendo así acceder y visualizar el contenido original fácilmente dentro de la propia interfaz de la herramienta. Todo esto facilita en gran medida la revisión por parte del analista, mostrando los resultados de forma clara y agrupada.

Etiquetado:

Además de la creación de artefactos, el módulo implementa un sistema de etiquetado automático mediante las funcionalidades nativas de Autopsy. A cada imagen procesada se le asignan una o más etiquetas según los objetos detectados por los modelos de visión por computador. Estas etiquetas (tags) permiten al perito forense clasificar, filtrar y localizar fácilmente evidencias visuales relevantes dentro del caso.

En la Figura 5.7, se observan las etiquetas generadas agrupadas jerárquicamente por tipo de objeto detectado (por ejemplo, 'dog', 'person', 'car'), incluyendo también categorías especiales para las detecciones textuales (CLIP_TXT) o búsqueda por imagen (CLIP_IMG). Aquellas imágenes que no han podido ser clasificadas se almacenan en la categoría "ImposibleClasificar", quedando igualmente disponibles para el investigador. Esta organización facilita la navegación entre resultados y permite llevar a cabo una última revisión manual en caso de ser necesario.

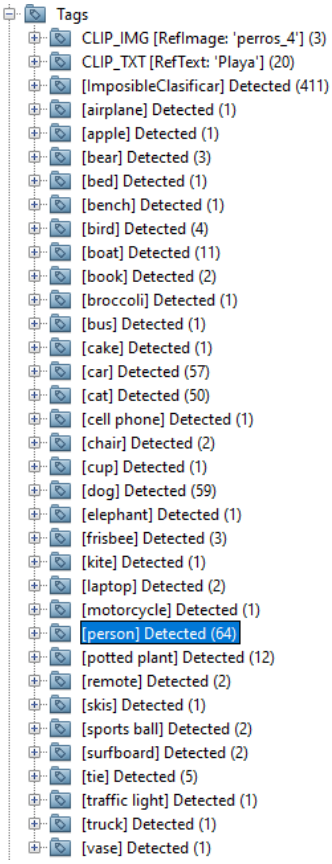


Figura 5.7: Organización de las categorías de etiquetas dentro del panel de resultados de Autopsy

Cabe destacar que una misma imagen puede contener varios objetos reconocibles de forma simultánea, como es el caso de la imagen "perros_14.jpg" que se aprecia en la Figura 5.9, donde aparece una persona y un perro. En esos casos, el sistema genera múltiples etiquetas asociadas a un único archivo, de modo que la imagen queda clasificada en distintas categorías a la vez. Por ejemplo, la imagen "perros_14.jpg" fue etiquetada tanto como 'person' como 'dog', cada una con su correspondiente nivel de confianza. Este comportamiento puede observarse en las Figuras 5.8 y 5.9, donde se muestra el artefacto y su clasificación dentro de la categoría 'person'. De esta misma forma, tendrá su entrada correspondiente en la categoría 'dog'.

Figura 5.8: Artefacto correspondiente a una imagen con las clases detectadas, usadas para su etiquetado

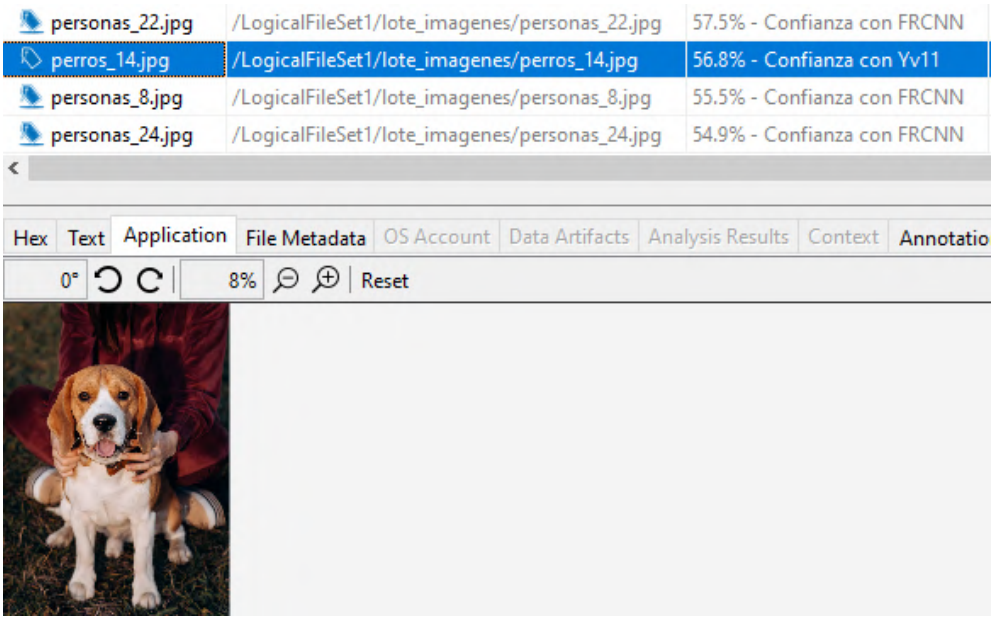


Figura 5.9: Artefactos de imágenes asociadas a una categoría concreta de etiquetas

La asignación de estas etiquetas se lleva a cabo en el propio módulo, dentro del método responsable del procesamiento de la respuesta del servidor. Concretamente, tras recibir la inferencia y validar sus datos, se recorren las detecciones obtenidas y se generan las etiquetas correspondientes con sus respectivos valores de confianza.

Este sistema de etiquetado automático complementa los artefactos forenses y mejora notablemente la capacidad del analista para localizar patrones visuales sin intervención manual, reforzando la trazabilidad y agilidad en entornos con grandes volúmenes de imágenes.

Sistema local de archivos:

Además de integrarse en la interfaz de Autopsy mediante artefactos y etiquetas, los resultados generados por el modelo también se guardan en el sistema de archivos local, específicamente dentro del directorio temporal asociado al caso activo. Durante la ejecución, el módulo organiza las imágenes procesadas en carpetas nombradas según las clases detectadas (por ejemplo, 'car', 'dog', 'person'), facilitando su revisión manual fuera del entorno gráfico de Autopsy. Esta estructura en disco permite al perito acceder directamente a los ficheros resultantes de cada ejecución y sirve como respaldo adicional de la información clasificada.

La Figura 5.10 ilustra la estructura previamente descrita.

```
C:\
└── Users\
    └── <tu_nombre_de_usuario>\
        └── AppData\
            └── Local\
                └── Temp\
                    └── Autopsy\
                        └── <nombre_del_caso>_YYYYMMDD_HHMMSS\
                            └── Temp\
                                ├── Detected_With_Yv11_Confianza_50pct\
                                │   ├── bird\
                                │   ├── dog\
                                │   ├── person\
                                │   └── ... (otras etiquetas detectadas)
                                │
                                └── Detected_With_Yv8_Confianza_80pct\
                                    ├── person\
                                    ├── laptop\
                                    └── backpack\
```

Figura 5.10: Estructura en disco de las imágenes procesadas, organizadas en carpetas por clases detectadas

5.2.7 Pruebas, logging y trazabilidad del módulo

Como fase final de este incremento se llevaron a cabo pruebas iniciales orientadas a validar el correcto funcionamiento del módulo desarrollado. Estas pruebas se centraron en aspectos básicos de interfaz y etiquetado, así como en la creación de artefactos y registros, permitiendo comprobar la estabilidad general de la aplicación tras el primer ciclo de desarrollo. En este contexto, resultó especialmente relevante la incorporación del sistema de logging, que contribuyó a la trazabilidad del proceso, la depuración y la auditoría posterior.

El logging se articuló en dos niveles: por un lado, mediante los registros internos de Autopsy, que permiten documentar mensajes dentro del entorno propio de la herramienta; y por otro, a través de un archivo de texto generado por el módulo, donde se anotan los principales eventos, configuraciones y posibles errores. Estos mecanismos ofrecieron al perito la posibilidad de conocer el detalle de cada ejecución, así como soporte en caso de fallos o mal funcionamiento. Además, fueron útiles para detectar errores en partes concretas del código, lo que facilitó su corrección de forma rápida y eficaz.

En síntesis, este incremento proporcionó una trazabilidad básica y suficiente para garantizar el funcionamiento inicial del sistema, consolidando las bases sobre las que se construyó el incremento posterior. La validación funcional completa, incluyendo pruebas integradas y métricas cuantitativas de precisión y rendimiento, se aborda más adelante en el Capítulo 6.

5.3 Incremento II: Servidor de inferencia

El servidor de inferencia constituye el segundo incremento del sistema, desarrollado siguiendo nuevamente un ciclo completo de análisis, diseño, codificación y pruebas, al igual que en el incremento anterior. En la fase de análisis se definieron los requisitos funcionales: recibir imágenes codificadas en Base64 [80] desde el módulo de Autopsy, seleccionar el modelo de IA adecuado, ejecutar la inferencia y devolver los resultados en un formato estructurado.

En la Tabla 5.2, se detallan las principales tareas que conforman este incremento.

Tareas	Descripción
T2.1	Estructura inicial del servidor FastAPI y configuración básica del entorno.
T2.2	Carga dinámica de modelos preentrenados y definición de las rutas de inferencia.
T2.3	Implementación de la inferencia para detección de objetos con procesamiento por lotes.
T2.4	Incorporación del modo de búsqueda por similitud de imagen.
T2.5	Incorporación del modo de búsqueda por similitud semántica mediante texto.
T2.6	Implementación del sistema de gestión de memoria con ajuste dinámico de lotes.
T2.7	Inicialización optimizada de modelos, cálculo del tamaño de lote adecuado y monitorización del uso de recursos.
T2.8	Integración de los incrementos y consolidación de la herramienta.

Tabla 5.2: Tareas del incremento II (servidor de inferencia)

Durante la fase de diseño se planificó la arquitectura del sistema, optando por un *backend* ligero en Python mediante FastAPI, con modelos de datos bien definidos (*ImageRequest*, *ImageResponse*) y un sistema de rutas específico para cada modo de inferencia. Asimismo, se diseñó su ejecución en contenedores Docker [68], lo que garantizó portabilidad, despliegue sencillo y escalabilidad en diferentes entornos forenses.

En la fase de codificación, cada una de las funcionalidades identificadas se implementó progresivamente: desde la estructura inicial del servidor y la carga dinámica de modelos preentrenados (YOLOv8/v11, Faster R-CNN, SSD y CLIP), hasta la integración de distintos modos de inferencia (detección de objetos, búsqueda por imagen o texto), el sistema de gestión de memoria con *batching dinámico* (ajuste automático del tamaño de lote en tiempo de ejecución

según memoria disponible, resolución de imagen y carga de peticiones) y la incorporación de tolerancia a errores para mejorar la robustez del servicio.

Finalmente, en la fase de pruebas se validó el comportamiento del servidor en distintos escenarios, comprobando la correcta recepción de imágenes, la ejecución de inferencias en los diferentes modos, la gestión eficiente de recursos y la capacidad de respuesta frente a errores de uso.

De este modo, el ciclo culminó con un incremento plenamente funcional, que dota al sistema de un servidor de inferencia integrado en un contenedor Docker, capaz de procesar imágenes desde el módulo cliente de Autopsy y devolver resultados fiables que enriquecen el análisis forense.

5.3.1 Fundamentos del servidor

Para dar soporte al análisis de imágenes desde el módulo de Autopsy, se ha desarrollado un servidor de inferencia independiente, diseñado como un microservicio RESTful utilizando el framework FastAP en Python. Este servidor actúa como núcleo de procesamiento, recibiendo peticiones desde el cliente, aplicando los modelos de IA seleccionados y devolviendo una respuesta estructurada con los resultados obtenidos.

La arquitectura se ha planteado de forma modular y desacoplada, lo que permite mantener separados los componentes del cliente y del servidor, facilitando su mantenimiento y escalabilidad. El servicio se ejecuta dentro de un contenedor Docker [68], lo que garantiza su portabilidad entre entornos y una rápida puesta en marcha.

Cabe señalar que, aunque la lógica del servidor está planteada como un componente separado, su despliegue puede realizarse en la misma máquina donde se ejecute Autopsy y el módulo desarrollado. Esta flexibilidad resulta especialmente útil en entornos de laboratorio o para investigadores individuales, al simplificar la configuración y reducir la necesidad de infraestructura adicional. No obstante, el desacoplamiento arquitectónico permite también migrar el servidor a otros entornos más potentes o distribuidos cuando sea necesario.

En cuanto a las tecnologías empleadas, el servidor utiliza PyTorch [82] como framework principal para el manejo de modelos de deep learning, junto con librerías complementarias como Torchvision [83], Pillow [66] y FastAPI [54].

La estructura del proyecto, representada en la Figura 5.11 está organizada de forma clara y modular, permitiendo separar las responsabilidades clave dentro del servidor. El archivo principal “main.py” actúa como punto de entrada de la API, gestionando las rutas y peticiones externas. Por su parte, la lógica de inferencia y el manejo de los modelos se encapsulan en un módulo auxiliar “ia_detect.py”, facilitando su mantenimiento y extensión. Además, el proyecto incluye los recursos necesarios para su ejecución en contenedor Docker, así como la definición de dependencias para asegurar la reproducibilidad del entorno:

```
/ServidorIA/
├── main.py            # Define la API y gestiona las rutas
├── ia_detect.py       # Lógica de selección y ejecución de modelos
├── Dockerfile         # Contenedor para facilitar despliegue
├── requirements.txt   # Dependencias del entorno
└── /models/           # Modelos preentrenados
```

Figura 5.11: Estructura jerárquica del servidor de inferencia

Este proceso inicial se asentó durante la tarea T2.1 recogida en la Tabla 5.2, en la que se codificaron las bases del servidor, la definición de datos y su estructura como microservicio REST [84].

5.3.2 Carga y gestión de modelos

La carga dinámica de los modelos de inferencia fue implementada en la tarea T2.2 recogida en la Tabla 5.2, incluyendo la gestión de rutas según tipo de modelo y su inicialización al arrancar el servidor.

Cuando el servidor se inicia, los modelos de visión por computador definidos en el archivo "ia_detect.py" son cargados dinámicamente en memoria y se mantienen disponibles en todo momento, a la espera de nuevas peticiones de análisis. Esta estrategia de precarga permite reducir la latencia durante la inferencia, ya que no es necesario cargar el modelo en cada ejecución, lo cual sería ineficiente en entornos donde se procesan lotes de imágenes de forma recurrente. El modelo se elige automáticamente en función del valor "model_flag" recibido en la petición. Según ese valor, el servidor ejecuta el modelo correspondiente sobre las imágenes enviadas, según el tipo de análisis solicitado.

La Figura 5.12 muestra la salida de consola tras iniciar el servidor y cómo queda a la espera de peticiones.

```
manuel@ubuntu2cprofile:~/servidor$ sudo docker logs -f ai-server-cpu
WARNING user config directory '/root/.config/Ultralytics' is not writeable, defaulting to '/tmp' or
CWD.Alternatively you can define a YOLO_CONFIG_DIR environment variable for this path.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at '/tmp/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see
https://docs.ultralytics.com/quickstart/#ultralytics-settings.
INFO:     Started server process [1]
INFO:     Waiting for application startup.
100%|██████████████████████████████████| 338M/338M [00:04<00:00, 83.4MiB/s]   a
WARNING:server:Medición valida para clip_txt: 18833408 bytes
WARNING:server:Medición max_by_ram para clip_txt: 1405 y cpu_cores: 32
WARNING:server:Medición valida para yv11: 15802368 bytes
WARNING:server:Medición max_by_ram para yv11: 1675 y cpu_cores: 32
Downloading: "https://download.pytorch.org/models/fasterrcnn_resnet50_fpn_coco-258fb6c6.pth" to
/root/.cache/torch/hub/checkpoints/fasterrcnn_resnet50_fpn_coco-258fb6c6.pth
100%|└└└
```

5.3.3 Manejo de peticiones y respuestas JSON

En la tarea T2.3 recogida en la Tabla 5.2 se implementó el manejo de peticiones mediante el endpoint `/process`, y la generación de respuestas JSON [81] estructuradas.

Una vez recibida la petición HTTP POST en el endpoint `/process` (procedimiento descrito en la sección 5.2.5), el servidor decodifica los datos JSON, interpreta los parámetros de configuración enviados (modelo, umbral, funcionalidad, imágenes y query) y ejecuta el modelo correspondiente sobre el lote de imágenes recibido.

Finalizada la inferencia, el servidor construye una respuesta estructurada en formato JSON, que contiene una lista de objetos resultado. Cada objeto de la respuesta representa una imagen analizada, identificada por su nombre, y contiene una lista de detecciones, donde cada una aporta información como la clase detectada, el grado de confianza y las coordenadas de la anotación.

El formato general de la respuesta viene representado en la Figura 5.13.

Para comprender el comportamiento, en las Secciones 5.3.4 y 5.3.5 se explicará paso a paso lo que sucede en el servidor desde que recibimos la petición, hasta que devolvemos el resultado.

Respuesta JSON

Tipo: JSON Response
Descripción: Resultado de la inferencia por imagen

JSON Payload:

```
[
  {
    "name": "img1.jpg",
    "preds": [
      {
        "label": "car",
        "score": 0.94,
        "box": [123, 45, 230, 150]
      },
      {
        "label": "person",
        "score": 0.81,
        "box": [15, 30, 88, 120]
      }
    ]
  },
  {
    "name": "img2.jpg",
    "preds": []
  }
]
```

Figura 5.13: Formato y estructura de la respuesta HTTP POST enviada al módulo

5.3.4 Manejo de imágenes y decodificación

Partiendo desde la recepción de la petición, el servidor comienza a decodificar las imágenes que llegan codificadas en formato Base64 [80]. Para garantizar un entorno seguro y limpio, cada ejecución crea un directorio temporal independiente, lo que evita conflictos entre sesiones y asegura que los archivos generados no interfieran entre sí.

Cada imagen se guarda en disco como un archivo físico en dicha carpeta temporal, utilizando nombres únicos derivados de su nombre original. Estos archivos temporales se gestionan internamente mediante una clase auxiliar del servidor, encapsulando tanto la ruta como la información contextual de cada imagen.

Estas imágenes ya decodificadas y organizadas son luego preparadas para su uso en el modelo correspondiente, aplicando las transformaciones necesarias antes de la inferencia, como redimensionado o normalización, según lo requiera cada arquitectura (YOLOv8/v11, Faster R-CNN, SSD o CLIP).

5.3.5 Estrategias de inferencia por modo de ejecución

Las estrategias de inferencia fueron desarrolladas progresivamente en las distintas tareas recogidas en la Tabla 5.2: la detección de objetos (mode=objects) en la tarea T2.3, la búsqueda por similitud de imagen (clip_img) en la T2.4, y la búsqueda por texto (clip_txt) en la T2.5.

El servidor está diseñado para adaptarse dinámicamente al tipo de análisis requerido según el parámetro `functionality` recibido en la petición. Este campo puede indicar una de las tres estrategias de inferencia disponibles: detección de objetos, búsqueda semántica por imagen o búsqueda semántica por texto (véase Tabla 5.3).

- **Modo `objects`:** Se utiliza cuando se solicita detección de objetos. El servidor selecciona el modelo correspondiente (YOLOv8, YOLOv11, SSD o Faster R-CNN) y lo aplica sobre cada imagen para identificar regiones con objetos de interés, devolviendo para cada uno su clase, puntuación de confianza y posición en la imagen.

- **Modo `clip_img`:** Este modo compara cada imagen del lote con una imagen de referencia también proporcionada en la petición. Se genera un valor de similitud para cada par de imágenes usando el modelo CLIP, lo que permite detectar contenido visual similar de forma robusta.

- **Modo `clip_txt`:** Similar al anterior, pero en lugar de usar una imagen como referencia, se utiliza una consulta textual. CLIP transforma tanto la imagen como el texto a un espacio común y calcula su similitud, identificando así las imágenes más relacionadas con la consulta semántica.

Modo	Entrada extra	Resultado
objects	Ninguna	Objetos detectados + bounding boxes
clip_img	Imagen de referencia	Similitud imagen–imagen
clip_txt	Texto descriptivo	Similitud imagen–texto

Tabla 5.3: Modos de ejecución del endpoint `/process`

Cada una de estas estrategias se encapsula en funciones independientes dentro del servidor, lo que permite mantener el código limpio y modular, y facilita la adición futura de nuevos modos de análisis.

5.3.6 Procesamiento por lotes y gestión de memoria

El sistema de cálculo dinámico del tamaño de lote (*batch size*) y la gestión eficiente de memoria fueron implementados en las tareas T2.6 y T2.7 recogidas en la Tabla 5.2, incluyendo el registro de uso de CPU y el control de ejecución progresiva en casos de fallo.

Para asegurar un uso eficiente de los recursos del sistema, el servidor realiza un cálculo dinámico del tamaño de lote permitido para cada modelo durante la fase de arranque. Este cálculo tiene en cuenta la memoria RAM disponible en el sistema y la huella de memoria (*footprint*) estimada que cada modelo requiere para procesar una única imagen. Esta estimación se obtiene mediante ejecuciones de prueba (*dummy*), controladas por el servidor para cada modelo, en las que se mide la memoria consumida con una entrada simulada.

Conocida la huella de memoria para cada modelo (`MEM_FOOTPRINT[flag]`), el tamaño de lote (*batch*) máximo seguro se fija como el mínimo entre el número de núcleos disponibles y el máximo de imágenes que caben en memoria.

```
batch_max = int(safe_ram // footprint)
batch_final = max(1, min(CPU_CORES, batch_max))
```

Listado 5.2: Cálculo del lote máximo y lote final seguro

Esta fórmula presente en el Listado 5.2 refleja el número máximo de imágenes que se pueden procesar simultáneamente para cada modelo sin superar el espacio de memoria disponible en un instante de tiempo, evitando así comprometer la estabilidad del sistema.

Durante la inferencia, el servidor intenta procesar las imágenes por lotes, pero si se produce un error de memoria (*RuntimeError*", comúnmente por OOM [86]), entra en una lógica de reducción progresiva del lote. En cada intento, el tamaño del batch se divide entre dos, hasta encontrar un tamaño que sea ejecutable, o se recurre a la inferencia secuencial si ni siquiera un lote de dos imágenes es posible.

Este mecanismo se aplica a todos los modos de inferencia (`objects, clip_img y`

`clip_txt`), asegurando que el servidor no se bloquee ante peticiones grandes y adaptándose dinámicamente a la carga y los recursos del sistema en tiempo real. Estos valores se ven reflejados en el panel (d) de la Figura 5.12.

5.3.7 Integración entre cliente y servidor

Una vez definidos y desarrollados los dos incrementos principales del sistema, descritos en las Secciones 5.2 y 5.3 y conforme a la tarea T2.8 recogida en la Tabla 5.2, su integración se realizó de forma directa gracias a la arquitectura modular adoptada desde el inicio. Ambos componentes permanecen desacoplados y se comunican exclusivamente mediante una interfaz REST, lo que simplifica el mantenimiento y facilita el despliegue en distintos entornos.

Este enfoque cliente-servidor ha resultado especialmente útil para sortear las limitaciones impuestas por Jython en Autopsy, permitiendo mantener la lógica de análisis fuera del entorno principal de la herramienta. De esta manera, el módulo actúa como cliente ligero que delega la carga computacional al servidor, manteniendo una interfaz limpia con el usuario y sin necesidad de gestionar directamente los modelos de IA.

La integración se apoya exclusivamente en el envío y recepción de mensajes HTTP bajo un formato JSON estandarizado. Esta decisión, junto con el uso de tecnologías compatibles y ampliamente adoptadas (FastAPI, Python, PyTorch), ha permitido alcanzar un sistema fácilmente extensible, capaz de incorporar nuevos modelos, funcionalidades o incluso migrar el servidor a entornos más potentes sin necesidad de modificar el cliente.

Adicionalmente, el servidor fue preparado para gestionar de forma controlada distintos tipos de errores que pueden surgir durante el procesamiento. En primer lugar, se implementó un control de validación en el propio endpoint: si la petición no contiene los campos esperados o no respeta el formato definido, se devuelve un código HTTP 400 (Bad Request), informando al cliente de la naturaleza del fallo. Por otro lado, durante la fase de inferencia, si se produce un error interno (por ejemplo, un fallo en la carga de los modelos, errores de decodificación o falta de memoria), la excepción se captura y se retorna un código HTTP 500 (Internal Server Error). De esta manera se garantiza que el cliente reciba un mensaje comprensible y que el sistema no se interrumpa de forma inesperada.

En todos los casos se emplearon los códigos de estado definidos en el estándar HTTP [73], lo que asegura compatibilidad y una interpretación coherente de las respuestas. Asimismo, se implementó un sistema de logging interno que registra mensajes relevantes en consola, como la carga de modelos, el cálculo dinámico del tamaño de lote, la llegada de peticiones y las excepciones detectadas. Esta trazabilidad resulta fundamental para la detección y resolución de problemas tanto durante el despliegue como en escenarios de uso concurrente.

5.3.8 Pruebas, gestión de errores y robustez

Como fase final de este incremento se llevaron a cabo pruebas orientadas a comprobar la solidez del servicio antes de su integración completa. Estas pruebas permitieron identificar incidencias tempranas y validar aspectos clave como el manejo de errores, la validación de entradas y la respuesta controlada ante fallos. Además, sirvieron para verificar el correcto funcionamiento del sistema de *logging* y de los mecanismos de tolerancia a errores previstos para el entorno productivo.

En síntesis, este incremento permitió dotar al servicio de una mayor robustez, con validaciones de entrada, gestión controlada de errores y una trazabilidad suficiente para garantizar estabilidad en entornos de análisis reales. Una vez consolidada esta base, en el Capítulo 6 se abordará la validación funcional del sistema completo, incluyendo pruebas integradas en escenarios representativos, así como la evaluación experimental de métricas de precisión, rendimiento y tiempos de respuesta.

5.4 Flujo de funcionamiento del sistema final

Tras el desarrollo de los dos incrementos principales (módulo de Autopsy y servidor de inferencia), resulta conveniente ofrecer una visión global que sintetice cómo interactúan sus componentes. El objetivo de esta sección es unificar lo expuesto previamente y mostrar, de forma integrada, el flujo completo de la herramienta en un escenario típico de uso forense.

Una vez integrados ambos incrementos, el sistema sigue el flujo representado en la Figura 5.14. En este diagrama de secuencia se muestra la interacción entre el módulo local y el servidor, abarcando desde la configuración inicial hasta la visualización de los resultados dentro de Autopsy.

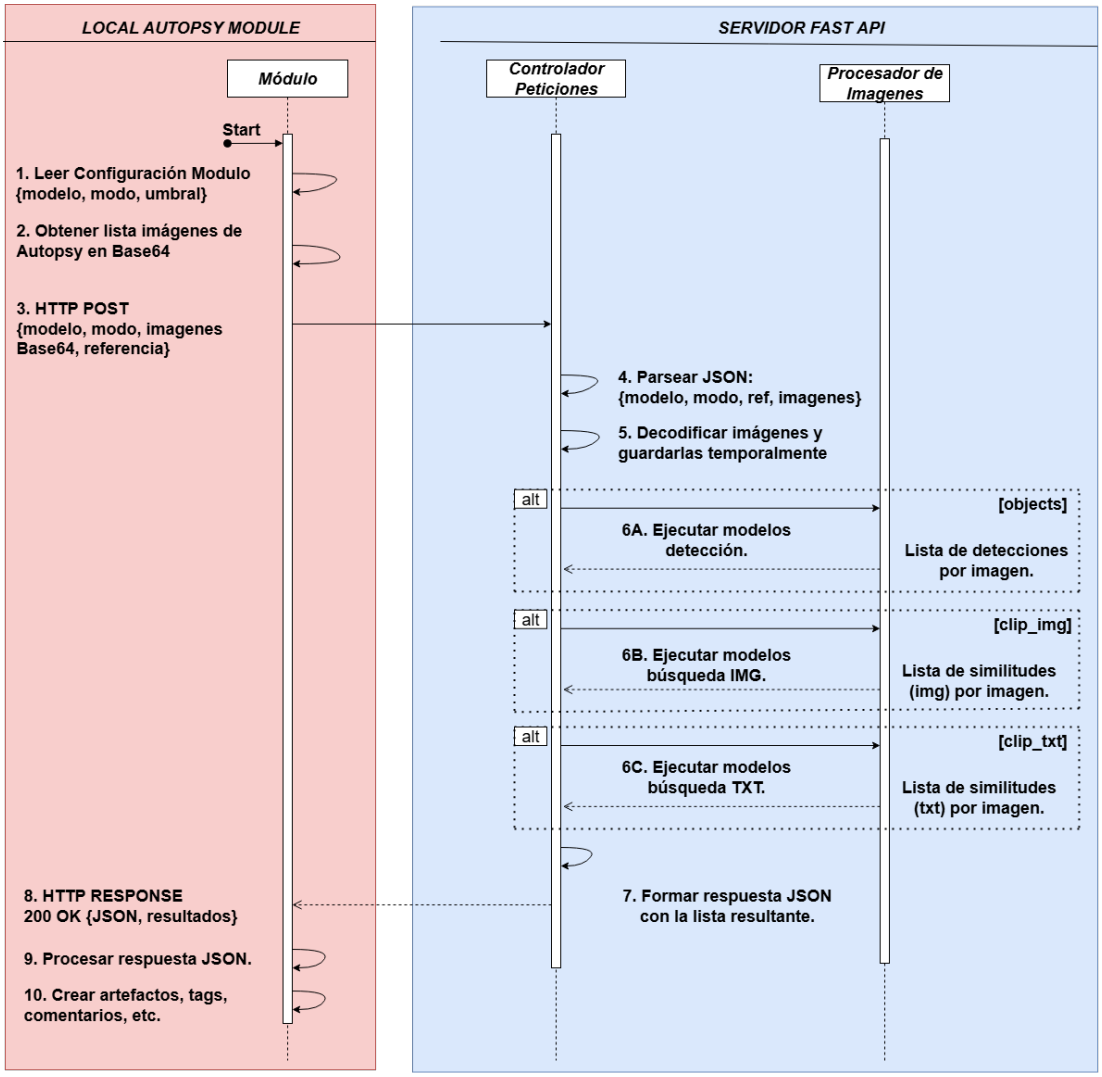


Figura 5.14: Diagrama de secuencia del funcionamiento completo de la herramienta

El proceso típico que sigue un investigador al utilizar la herramienta se resume en las siguientes fases:

1. El módulo lee la configuración del usuario, donde se indican parámetros como el umbral de confianza de los resultados, el modelo elegido y, en su caso, las variables de referencia necesarias para el análisis.

2. El módulo obtiene las imágenes del dispositivo de almacenamiento a partir de la información que proporciona Autopsy. Estas se formatean, procesan y preparan para su envío al servidor.

3. Las imágenes se agrupan y envían en un único lote mediante una petición al servidor,

junto con las configuraciones definidas por el usuario.

4. El servidor recibe la petición, interpreta el JSON y determina el modo de ejecución y las imágenes a procesar.
5. Las imágenes se decodifican y se preparan para su procesamiento en el bloque de código correspondiente.
6. Se ejecuta la inferencia de los modelos sobre las imágenes.
7. Se construye la respuesta con los resultados obtenidos y se envía de vuelta al módulo en Autopsy.
8. El módulo recibe la respuesta.
9. Los resultados son procesados y estructurados.
10. Finalmente, se representan de forma visual en la interfaz de Autopsy, integrándose con sus mecanismos nativos de visualización y anotación.

Este flujo garantiza que el analista pueda visualizar de forma clara y ordenada los resultados de la inferencia realizada sobre las imágenes extraídas del dispositivo de almacenamiento, consolidando así la integración entre Autopsy y los modelos de IA.

Para completar la descripción técnica, el procedimiento de instalación, configuración y puesta en marcha de la herramienta se presenta de forma detallada en el Apéndice A, que se ofrece como manual de instalación independiente.

Capítulo 6

Evaluación

Con el desarrollo del sistema completado, resulta imprescindible comprobar su correcto funcionamiento y medir su efectividad en condiciones reales de uso. Este capítulo recoge, en primer lugar, la validación funcional de las distintas características implementadas, ilustrando cómo responden ante casos prácticos de análisis forense. A continuación, se presenta la evaluación técnica basada en métricas cuantitativas de precisión, rendimiento y tiempo de respuesta, con el fin de valorar el grado de cumplimiento de los objetivos definidos y la utilidad real de la herramienta en escenarios forenses.

6.1 Validación funcional

Las pruebas realizadas sobre el sistema completo recogen los resultados obtenidos en cada ciclo de vida de los incrementos desarrollados. A continuación, se presenta un resumen de las validaciones de todas las funcionalidades de manera conjunta, acompañado de ejemplos reales de ejecución para cada una de ellas. Con esta sección se pretende ilustrar la experiencia del investigador, mostrando cómo recibe y analiza la información generada por la herramienta.

Para este fin se diseñó un conjunto de pruebas reducido en el que se ejecutó el módulo de análisis activando en cada ocasión uno de los modelos disponibles de forma individual: YOLOv8, YOLOv11, Faster R-CNN, SSD y CLIP (en ambos modos de operación).

Esta parte actúa como una validación funcional del sistema, comprobando que la arquitectura cliente-servidor se comporta correctamente en todos los modos de análisis implementados y demostrando la utilidad de la herramienta a la hora de organizar los resultados.

6.1.1 Pruebas de detección de objetos

Se ha utilizado como fuente de datos una unidad USB, que contiene cinco imágenes seleccionadas manualmente desde el repositorio público Unsplash [87]. Cada una de ellas corresponde a un escenario de prueba distinto (entorno de escritorio, conjunto de utensilios, animal,

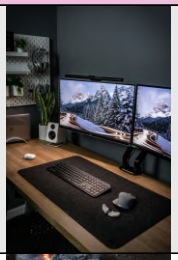
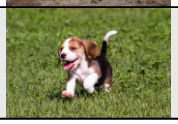
Imagen	YOLOv8	YOLOv11	Faster R-CNN	SSD
	potted plant (0.492), keyboard (0.844), chair (0.470), mouse (0.499)	keyboard (0.842), mouse (0.681)	potted plant (0.539), keyboard (0.562), tv (0.291), suitcase (0.226), chair (0.258), couch (0.283), vase (0.295), laptop (0.260), boat (0.207), mouse (0.978)	keyboard (0.157)
	bottle (0.800), bowl (0.602), person (0.668)	bottle (0.665), bowl (0.632), person (0.717)	dining table (0.329), bottle (0.369), bowl (0.260), person (0.460), cake (0.219)	Imposible clasificar con este modelo
	clock (0.860), knife (0.805), fork (0.458), carrot (0.651)	clock (0.770), fork (0.829), knife (0.728)	dining table (0.533), clock (0.584), orange (0.947), knife (0.534), fork (0.990)	clock (0.137)
	person (0.855)	person (0.719)	car (0.429), person (0.642)	truck (0.112), motorcycle (0.146)
	dog (0.933)	dog (0.914)	dog (0.997)	dog (0.111)

Tabla 6.1: Comparación de las detecciones realizadas por los modelos YOLOv8, YOLOv11, Faster R-CNN y SSD sobre cinco imágenes representativas

escena con personas y vehículo, y una mesa con comida).

Una vez cargadas las imágenes en el caso de Autopsy, ejecutamos el análisis completo activando uno a uno los distintos modelos disponibles. El objetivo aquí no era medir el rendimiento o precisión (propósito abordado en la Sección 6.2), sino verificar que todo el flujo cliente-servidor funciona correctamente: desde la preparación de la petición, hasta la recepción de resultados y su representación visual dentro de la interfaz forense.

La Tabla 6.1 recoge un resumen de los resultados obtenidos para cada imagen. Se aprecia que todos los modelos produjeron detecciones válidas, aunque con comportamientos diferentes.

Toda la información presentada ha sido extraída directamente desde la interfaz de Autopsy, sin necesidad de pasos adicionales. Los artefactos creados por el módulo no sólo almacenan los objetos detectados y sus valores de confianza, sino que también permiten navegar rápidamente por las imágenes asociadas.En todos los casos, los resultados fueron correctamente devueltos por el servidor de inferencia y almacenados en Autopsy como artefactos etiquetados, demostrando la integración efectiva de los modelos en el flujo de trabajo habitual.

En las imágenes recogidas en la Figura 6.1 se puede ver un ejemplo de cómo se visualizan

Detected With Yv11 Confianza 55pct

Table | Thumbnail | Summary

Source Name	S	C	O	Source Type	Comment
4.jpg				File	Detections: person(0.719)
5.jpg				File	Detections: bottle(0.665), bowl(0.632), person(0.717)
1.jpg				File	Detections: keyboard(0.842), mouse(0.681)
2.jpg				File	Detections: clock(0.770), fork(0.829), knife(0.728)
3.jpg				File	Detections: dog(0.914)

Detected With SSD Confianza 10pct

Table | Thumbnail | Summary

Source Name	S	C	O	Source Type	Comment
1.jpg				File	Detections: keyboard(0.157)
2.jpg				File	Detections: clock(0.137)
3.jpg				File	Detections: dog(0.111)
4.jpg				File	Detections: truck(0.112), motorcycle(0.146)
5.jpg				File	Imposible Clasificar usando SSD con umbral 10%

Detected With Yv8 Confianza 40pct

Table | Thumbnail | Summary

Source Name	S	C	O	Source Type	Comment
1.jpg				File	Detections: potted plant(0.492), keyboard(0.844), chair(0.470), mouse(0.499)
2.jpg				File	Detections: clock(0.860), knife(0.805), fork(0.458), carrot(0.651)
3.jpg				File	Detections: dog(0.933)
4.jpg				File	Detections: person(0.855)
5.jpg				File	Detections: bottle(0.800), bowl(0.602), person(0.668)

Detected With FRCNN Confianza 20pct

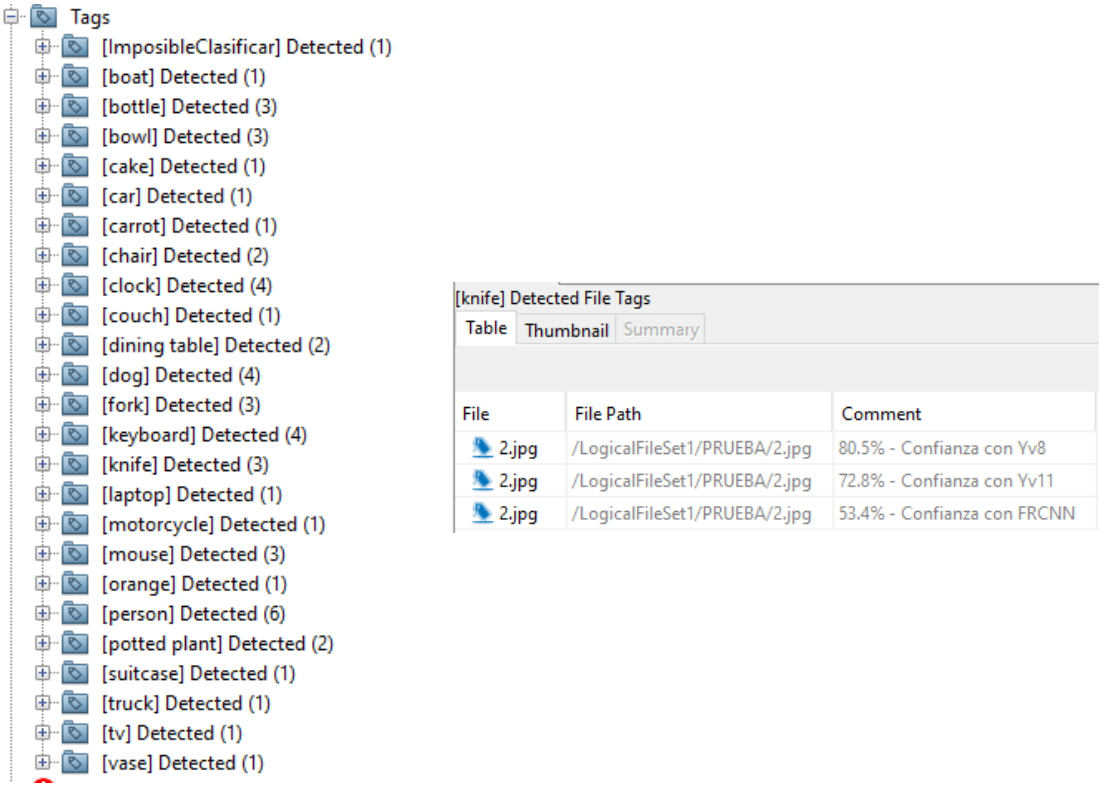
Table | Thumbnail | Summary

Source Name	S	C	O	Source Type	Comment
1.jpg				File	Detections: potted plant(0.539), keyboard(0.562), tv(0.291), suitcase(0.226), chair(0.258), couch(0.283), vase(0.295), laptop(0.260), boat(0.207), mouse(0.978)
2.jpg				File	Detections: dining table(0.533), clock(0.584), orange(0.947), knife(0.534), fork(0.990)
3.jpg				File	Detections: dog(0.997)
4.jpg				File	Detections: car(0.429), person(0.642)
5.jpg				File	Detections: dining table(0.329), bottle(0.369), bowl(0.260), person(0.460), cake(0.219)

Figura 6.1: Resultados de la ejecución de modelos de detección de objetos en Autopsy, con ejemplos de artefactos etiquetados en la interfaz.

estos mismos resultados descritos en la Tabla 6.1, dentro del entorno de trabajo de Autopsy. Esta representación gráfica resulta muy útil para que el analista pueda revisar, comparar y filtrar hallazgos de forma ágil. Cabe destacar que, al hacer clic sobre cualquiera de los artefactos generados, Autopsy permite previsualizar la imagen asociada directamente desde su interfaz, lo que agiliza enormemente el análisis y la revisión del contenido.

En las imágenes recogidas en la Figuras 6.2 se observa cómo el sistema genera automáticamente las etiquetas correspondientes a cada objeto detectado. Esta funcionalidad ofrece al investigador un método visual e intuitivo para localizar evidencias relevantes dentro del



(a) Todas las categorías (b) Categoría concreta

Figura 6.2: Ejemplos de etiquetas generadas automáticamente por el sistema en la interfaz de Autopsy

conjunto de imágenes analizadas.

La Figura 6.2a muestra el listado completo de categorías creadas, que permiten filtrar rápidamente el contenido en función de las etiquetas asignadas. Si el analista desea centrarse en una categoría concreta, como “knife” (véase Figura 6.2b), puede acceder directamente a la carpeta asociada, donde se agrupan todas las imágenes relacionadas con esa detección.

Cada archivo dentro de la carpeta incluye información adicional sobre el modelo que identificó el objeto y el nivel de confianza alcanzado. Esta trazabilidad facilita ordenar los resultados por modelo o por porcentaje de confianza, lo que agiliza el análisis detallado y la comparación de evidencias en el entorno forense.

Por último, en la Figura 6.3, correspondiente a la salida del servidor, se confirma el envío de las respuestas hacia Autopsy, junto con el cálculo dinámico del tamaño de lote empleado para cada modelo, lo que evidencia la robustez del proceso de comunicación.

```
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
WARNING:server:Nueva medición para yv8: 39345 batch-bytes and CPU: 32, nueva batch size: 32
INFO:     10.29.23.232:59776 - "POST /process HTTP/1.1" 200 OK
WARNING:server:Nueva medición para yv11: 1542 batch-bytes and CPU: 32, nueva batch size: 32
INFO:     10.29.23.232:59864 - "POST /process HTTP/1.1" 200 OK
WARNING:server:Nueva medición para frcnn: 327 batch-bytes and CPU: 32, nueva batch size: 32
INFO:     10.29.23.232:59890 - "POST /process HTTP/1.1" 200 OK
WARNING:server:Nueva medición para ssd: 640 batch-bytes and CPU: 32, nueva batch size: 32
INFO:     10.29.23.232:59951 - "POST /process HTTP/1.1" 200 OK
```

Figura 6.3: Salida del servidor mostrando el envío de respuestas hacia Autopsy y el cálculo dinámico del tamaño de lote usado para cada modelo

6.1.2 Pruebas de búsqueda por similitud de texto

Para validar el correcto funcionamiento del modo de búsqueda por similitud mediante texto (CLIP-TXT), se preparó un conjunto de prueba compuesto por tres imágenes que muestran a distintas personas en contextos cotidianos. En una de ellas, una mujer sostiene claramente un cuchillo, lo que permite comprobar si el sistema es capaz de detectar este detalle a partir de una consulta textual precisa.

La Figura 6.4 muestra la configuración seleccionada, en la que se definió como texto de referencia “Mujer con cuchillo” y un umbral de confianza mínimo del 20%.

Las tres imágenes utilizadas en la prueba se ilustran en la Figura 6.5.

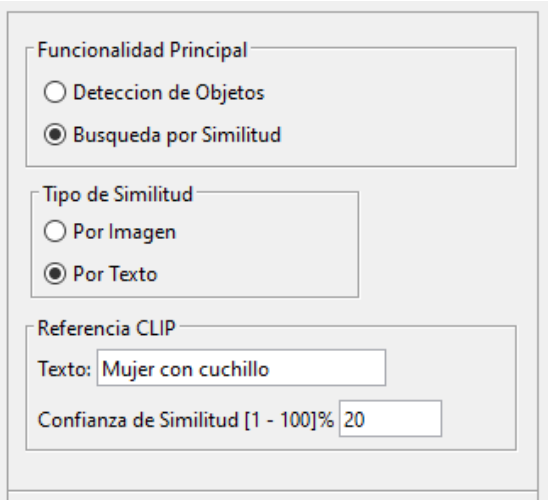


Figura 6.4: Configuración seleccionada para la prueba de búsqueda por similitud de texto

(a) HombreYOrdenador.png

(b) MujerYCuchillo.png

(c) MujerYOrdenador.png

Figura 6.5: Imágenes utilizadas en la prueba de búsqueda por similitud de texto

Tras la ejecución del modelo se obtuvieron los resultados que se resumen en la Tabla 6.2. La imagen MujerYCuchillo.png 6.5b alcanzó la mayor similitud (26.6%), seguida de MujerYOrdenador.png 6.5c (20.2%), mientras que HombreYOrdenador.png 6.5a no superó el umbral definido.

Estos valores muestran que el sistema priorizó correctamente la imagen más relevante para la consulta y aplicó de forma adecuada el umbral de confianza para descartar resultados irrelevantes.

En la Figura 6.6 se aprecia cómo se presenta esta misma información de manera clara y organizada dentro de la propia interfaz de Autopsy.

Imagen	Similitud (%)	Resultado
MujerYCuchillo.png	26.6	Válida
MujerYOrdenador.png	20.2	Válida
HombreYOrdenador.png	< 20	No Válida

Tabla 6.2: Resultados de la prueba de búsqueda por similitud de texto

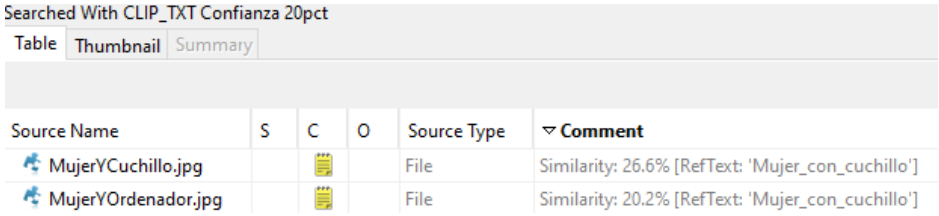
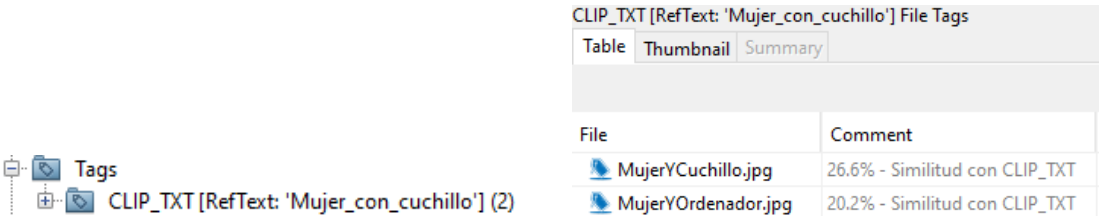


Figura 6.6: Presentación de los resultados de la prueba búsqueda de similitud de texto dentro de la interfaz de Autopsy

Finalmente, en la Figura 6.7 se muestra la etiqueta creada automáticamente, asociada a la búsqueda realizada.



(a) Categoría generada (b) Imágenes etiquetad

Figura 6.7: Etiquetas generadas automáticamente como resultado de la prueba búsqueda de similitud de texto dentro de la interfaz de Autopsy

Aunque las puntuaciones absolutas puedan parecer bajas, distintos estudios sobre CLIP y otros modelos de recuperación semántica indican que umbrales entre el 20% y el 30% son suficientes para capturar asociaciones relevantes en tareas sin ejemplos previos (zero-shot) [88], [89]. Esto refleja que el modelo no evalúa coincidencias exactas, sino correspondencias conceptuales, lo que otorga significado práctico incluso a valores moderados.

En definitiva, la prueba demuestra que CLIP-TXT fue capaz de identificar la relación semántica entre el texto "mujer con cuchillo" y el contenido visual, activando correctamente las imágenes pertinentes. Asimismo, pone de manifiesto cómo pequeñas diferencias en los valores de similitud pueden condicionar la inclusión o exclusión de resultados, reforzando la importancia de seleccionar umbrales adecuados según el contexto del análisis forense.

6.1.3 Pruebas de búsqueda por similitud de imagen

Se diseñó una prueba destinada a evaluar la funcionalidad de búsqueda por similitud entre imágenes. Para ello se utilizaron cuatro fotografías de autoría propia, todas ellas representando perros en distintos entornos. Dos de estas imágenes correspondían al mismo animal, mientras que las otras dos mostraban ejemplares diferentes, tanto en raza como en contexto visual. El propósito era comprobar si el sistema, al usar una de las imágenes como referencia, era ca-

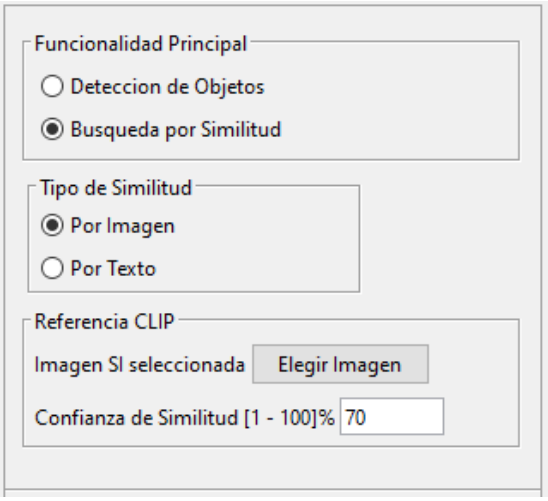


Figura 6.8: Configuración seleccionada para la prueba de búsqueda por similitud de imagen

paz de identificar correctamente aquellas más similares dentro del conjunto, priorizando las que contenían al mismo individuo. Esta capacidad resulta especialmente valiosa en escenarios forenses, donde la detección de coincidencias visuales puede aportar información clave.

La Figura 6.8 muestra la configuración de la herramienta para este caso, empleando la imagen 1.A.png como referencia visual y un umbral de similitud del 70%.

En la Figura 6.9 se presentan las cuatro imágenes utilizadas en la prueba de búsqueda por similitud de imagen.

La imagen de referencia 1.A.png 6.9a obtuvo una coincidencia perfecta (100%), seguida de su duplicado 1.B.png 6.9b con un 86.3%. Las otras dos imágenes, 2.png 6.9c y 3.png 6.9d tam-

(a) Imagen 1.A.png (b) Imagen 1.B.png (c) Imagen 2.png (d) Imagen 3.png

Figura 6.9: Conjunto de imágenes utilizadas en la prueba de búsqueda por similitud de imagen

Imagen	Similitud (%)	Resultado
1.A.jpeg	100	Válida (referencia)
1.B.jpeg	86.3	Válida
2.jpeg	70.3	Válida
3.jpeg	76.4	Válida

Tabla 6.3: Resultados de la prueba de búsqueda por similitud de imágenes

bién superaron el umbral mínimo, aunque con valores inferiores (70.3% y 76.4%), lo que refleja cierta similitud visual sin llegar a identificar al mismo individuo. Los resultados obtenidos se resumen en la Tabla 6.3.

En la Figura 6.10 se muestra cómo esta información se organiza y visualiza directamente dentro de la interfaz de Autopsy.

Finalmente, la Figura 6.11 muestra el resultado del etiquetado asociado a la búsqueda, donde las imágenes relevantes quedan organizadas en sus respectivas categorías para facilitar la investigación.

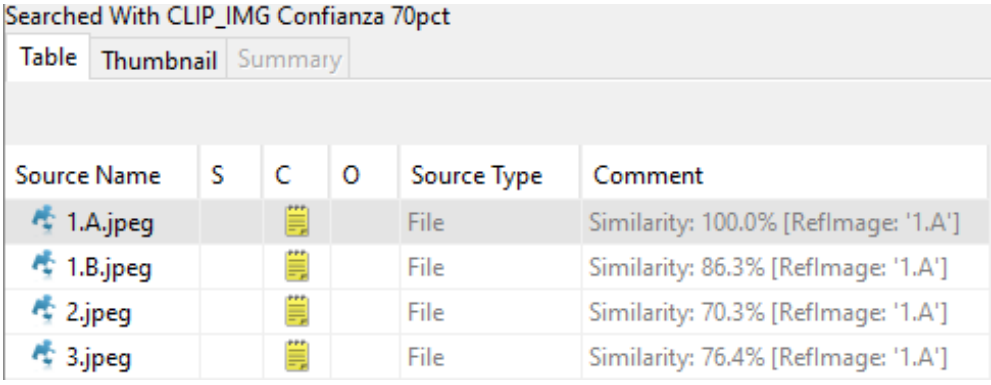
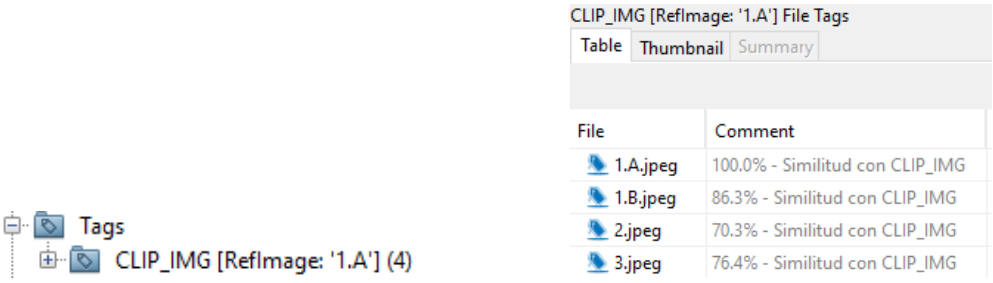


Figura 6.10: Presentación de los resultados de similitud directamente en la interfaz de Autopsy



(a) Categoría generada (b) Imágenes etiquetad

Figura 6.11: Etiquetas generadas automáticamente como resultado de la prueba búsqueda de similitud de imágenes dentro de la interfaz de Autopsy

6.2 Evaluación del rendimiento

El propósito de esta sección es dar cumplimiento al objetivo número 3, descrito en la Sección 1.2, demostrando que la solución desarrollada aporta un valor significativo en el ámbito de la informática forense. Para lograr esto se evalúan métricas cuantitativas como la precisión, el rendimiento y el tiempo de respuesta, con el fin de verificar la efectividad del módulo y de los modelos integrados.

Esta evaluación permite comprobar tanto la fiabilidad operativa del sistema como su capacidad para generar resultados útiles en escenarios reales de análisis forense. De este modo, se valida que la herramienta no solo satisface los requisitos funcionales definidos, sino que también responde adecuadamente a las exigencias prácticas de los entornos en los que está destinada a ser utilizada.

6.2.1 Metodología de evaluación

Con el objetivo de validar la efectividad operativa del sistema desarrollado, se ha diseñado una estrategia de evaluación basada en métricas cuantificables y orientadas al contexto forense. Esta metodología permite analizar si la herramienta responde de manera robusta, precisa y eficiente ante distintos escenarios de análisis visual, garantizando su utilidad práctica en entornos reales.

La evaluación se ha estructurado en torno a tres dimensiones principales:

- **Precisión:** entendida como la capacidad de los modelos para detectar correctamente objetos relevantes o establecer similitudes válidas entre imágenes y/o texto. Para este aspecto se tomarán como referencia las métricas oficiales publicadas por los desarrolladores de cada modelo (YOLOv8, SSD, Faster R-CNN, CLIP), obtenidas sobre conjuntos de datos de uso generalizado como COCO [49] o ImageNet [88].

- **Rendimiento:** medido a través del tiempo necesario para procesar lotes de imágenes de distintos tamaños, tanto en tareas de detección como de similitud. Estos tiempos se registran automáticamente desde el servidor, lo que permite evaluar la eficiencia computacional y la escalabilidad del sistema bajo diferentes cargas de trabajo.
- **Comparación con el análisis humano:** se estima el tiempo que requeriría un perito para realizar manualmente las mismas tareas que el sistema, con el fin de contextualizar los beneficios de la automatización. Esta estimación se apoya en observación directa y fuentes del ámbito forense.

Las pruebas se han realizado sobre conjuntos de imágenes controlados, manteniendo constantes parámetros como el umbral de confianza y el entorno de ejecución (CPU). De cada ejecución se han recogido resultados relevantes (tiempos, detecciones, coincidencias, similitudes y artefactos generados) para su análisis cuantitativo y cualitativo posterior, lo que permite extraer conclusiones sobre la solidez, eficiencia y aplicabilidad del sistema en un contexto real.

Todas las pruebas descritas en los apartados siguientes se ejecutaron sobre los equipos detallados en la Sección 4.3. Esta referencia resulta esencial para interpretar los valores de rendimiento, dado que la carga computacional varía según el hardware disponible.

6.2.2 Evaluación de precisión

Para evaluar la precisión del sistema propuesto, se ha optado por una metodología basada en dos pilares: por un lado, el uso de métricas cuantitativas oficiales extraídas de los estudios y benchmarks de cada modelo, y por otro, la validación funcional a través de pruebas reales, que demuestran su comportamiento sobre imágenes concretas en escenarios de análisis forense.

Precisión según benchmarks oficiales

Los modelos empleados han sido previamente entrenados y evaluados en datasets estandarizados y ampliamente aceptados por la comunidad científica, como COCO (Common Objects in Context) [49] o ImageNet [88]. En dichos entornos se utilizan métricas consolidadas, entre las que destacan:

- **mAP (mean Average Precision):** empleado en la evaluación de los modelos de detección de objetos (YOLO, SSD, Faster R-CNN), mide la precisión promedio considerando distintos umbrales de intersección sobre la unión *Intersection over Union*(IoU) [90].
- **Top-1 / Top-5 accuracy y similaridad coseno:** utilizadas en el caso de CLIP, orientadas a evaluar la recuperación de imágenes o el emparejamiento texto–imagen. Estas métricas permiten cuantificar la capacidad del modelo para seleccionar la respuesta correcta en la primera posición (Top-1), dentro de las cinco primeras opciones (Top-5), o

Modelo	Dataset base	Métrica usada	Precisión aproximada
YOLOv8	COCO	mAP@0.5:0.95	∼0.54
YOLOv11	COCO	mAP@0.5:0.95	∼0.58
SSD300	COCO	mAP@0.5:0.95	∼0.44
Faster R-CNN	COCO	mAP@0.5:0.95	∼0.60
CLIP ViT-B/32	ImageNet / COCO	Top-1 Accuracy	∼0.38 (zero-shot)

Tabla 6.4: Resumen de métricas oficiales reportadas para los modelos utilizados en el proyecto

a través de la proximidad en el espacio semántico mediante el cálculo de la similaridad coseno.

La Tabla 6.4 resume los valores aproximados de precisión reportados en la documentación oficial y en trabajos de referencia [13, 14, 23, 61, 62]:

Estos valores muestran que los modelos ofrecen un equilibrio razonable entre velocidad y precisión, y que resultan adecuados para la detección y clasificación de elementos en contextos visuales diversos. En el caso de CLIP, aunque las tasas de acierto absoluto son más modestas, su capacidad para capturar relaciones semánticas entre texto e imagen lo convierte en una herramienta especialmente útil en búsquedas abiertas sin categorías predefinidas.

Validación funcional mediante pruebas objetivas

Además de las métricas teóricas, se llevó a cabo una verificación práctica del comportamiento de los modelos sobre un conjunto reducido de imágenes representativas (véase Sección 6.1). El objetivo fue comprobar que la herramienta mantiene un rendimiento coherente en escenarios realistas de análisis forense.

En estas pruebas se constató que el sistema es capaz de:

- Detectar correctamente múltiples objetos presentes en escenas variadas (vehículos, personas, cuchillos).
- Priorizar imágenes según su similitud semántica con un texto dado (por ejemplo, “mujer con cuchillo”).
- Identificar imágenes visualmente similares, reconociendo incluso al mismo animal en fotografías diferentes.

Aunque estos resultados no son directamente cuantificables mediante métricas estándar como mAP, evidencian un funcionamiento consistente y útil del sistema en tareas de análisis forense visual. De este modo, las pruebas prácticas complementan los benchmarks oficiales y

confirman que el módulo desarrollado ofrece un nivel de precisión suficiente para su aplicación en contextos reales.

6.2.3 Evaluación de rendimiento

Para analizar el rendimiento de los modelos integrados en el sistema, se ejecutó una prueba sobre un conjunto fijo de 40 imágenes, midiendo el tiempo total de procesamiento de principio a fin. Esta evaluación buscó ofrecer una visión comparativa de la eficiencia de cada modelo, expresada en duración total del análisis y tiempo medio por imagen.

Procedimiento de medición:

Los tiempos se registraron directamente en Autopsy, desde que el usuario inicia el análisis (START) hasta la recepción de resultados y generación de artefactos. Para automatizar este registro, se incluyó un bloque de código en los métodos `startUp()` y `shutDown()` del cliente, que calcula tanto la duración total como el promedio por imagen (véase Listado 6.1).

```
# Dentro del método startUp(), al iniciar el análisis
self.start_time = time.time()

# Dentro del método shutDown(), al finalizar el análisis
end_time = time.time()
elapsed = end_time - self.start_time
avg_per_image = elapsed / 40

# Obtención del modelo activo desde la configuración local
model = self.local_settings.getSetting("ai_model_flag") or "Desconocido"

# Ruta de ejemplo donde se guardará el registro
log_path = os.path.expanduser(r"C:\logs\tiempos_de_ejecucion.txt")

# Escritura de los resultados en el archivo de log
with open(log_path, "a", encoding="utf-8") as f:
    f.write("\n==== Evaluación de rendimiento ====\n")
    f.write(f"Modelo: {model}\n")
    f.write(f"Inicio: {self.start_time:.2f} s\n")
    f.write(f"Fin: {end_time:.2f} s\n")
    f.write(f"Duración total: {elapsed:.2f} s\n")
    f.write(f"Imágenes: 40\n")
    f.write(f"Tiempo promedio: {avg_per_image:.4f} s\n")
```

Listado 6.1: Registro de tiempos de ejecución y rendimiento del sistema

Resultados obtenidos:

La Tabla 6.5 recoge los tiempos medidos para los seis modelos disponibles.

Modelo	Tiempo total (s)	Tiempo medio por imagen (s)
YOLOv8	8.96	0.2241
YOLOv11	9.30	0.2324
Faster R-CNN	70.71	1.7678
SSD	21.07	0.5267
CLIP (Imagen)	7.56	0.1890
CLIP (Texto)	8.04	0.2009

Tabla 6.5: Tiempos de ejecución totales y promedios por imagen de los distintos modelos integrados en el sistema

6.2.4 Comparación con análisis manual

Para valorar la eficiencia del sistema, se realizó una comparación con el tiempo estimado que requeriría un análisis manual equivalente llevado a cabo por un perito forense.

Con base en observación directa durante el desarrollo del proyecto y en la literatura del sector, se fijó una referencia de 20 segundos por imagen como valor razonable y conservador para una revisión visual rápida sin tareas adicionales de documentación o etiquetado. Este criterio se apoya en estudios como el de Roth (2025), quien señala que el análisis humano en informática forense resulta lento, propenso a errores y con un rendimiento decreciente a medida que aumenta el volumen de información [91].

Aplicando esta estimación al conjunto de 40 imágenes utilizado en las pruebas, el tiempo total requerido por un humano sería:

- Tiempo estimado para un análisis manual:
  - 40 imágenes × 20 segundos = 800 segundos
  - 800 segundos ≈ 13 minutos y 20 segundos

Este valor sirve de referencia para contrastar el rendimiento de los modelos automáticos integrados en el sistema, cuyos tiempos de ejecución oscilaron entre 7,5 y 70 segundos. La Tabla 6.6 resume dicha la comparación.

Método	Tiempo total (40 imágenes)	Mejora respecto a análisis manual
Humano (estimado)	∼800 s (13 min 20 s)	N/A
YOLOv8	∼8.96 s	∼89x más rápido
YOLOv11	∼9.30 s	∼86x más rápido
Faster R-CNN	∼70.71 s	∼11x más rápido
SSD	∼21.07 s	∼38x más rápido
CLIP (Imagen)	∼7.56 s	∼106x más rápido
CLIP (Texto)	∼8.04 s	∼99x más rápido

Tabla 6.6: Comparación del tiempo de análisis manual frente al uso de los distintos modelos integrados en el sistema

Incluso el modelo más lento ofrece una mejora de rendimiento significativa en comparación con la revisión manual estimada, lo que demuestra la eficacia del sistema como herramienta de cribado automático en entornos reales, aliviando la carga de trabajo del personal forense y permitiendo priorizar rápidamente las imágenes que requieren atención humana.

Capítulo 7

Resultados

En este capítulo se presentan de forma sintética los principales resultados obtenidos a lo largo del desarrollo del trabajo. Dado que muchos de ellos ya han sido descritos en detalle en los apartados de Desarrollo (Capítulo 5) y Evaluación (Capítulo 6), aquí se ofrece una visión global orientada a mostrar en qué medida se han alcanzado los objetivos planteados. Para cada uno de ellos se resume brevemente el logro correspondiente y se remite, cuando es necesario, a los apartados donde se encuentra la explicación completa.

Estos resultados reflejan tanto el cumplimiento de los objetivos iniciales como los avances obtenidos en su validación práctica. El proyecto responde a un reto central en la informática forense: el análisis de grandes volúmenes de imágenes digitales en dispositivos de almacenamiento, una tarea cada vez más compleja que los métodos manuales no pueden abordar de manera eficiente.

La herramienta desarrollada pretende asistir al perito en la localización temprana de información visual relevante. Su propósito es reducir los tiempos de revisión y aliviar la carga de trabajo, sin alterar el papel central del analista en la investigación. Para ello, automatiza tareas como la clasificación, la detección de objetos y la búsqueda semántica, manteniendo siempre la trazabilidad de los hallazgos y respetando la cadena de custodia de la evidencia digital.

El resultado es una herramienta que combina modelos de inteligencia artificial de visión por computador con un diseño orientado a su integración nativa en Autopsy, uno de los entornos más utilizados en laboratorios de análisis forense. Su arquitectura modular se organiza en dos componentes principales: un módulo de ingestión, integrado como plugin en Autopsy, y un servidor de inferencia independiente que ejecuta los modelos de IA. Esta separación entre la gestión de evidencias y el procesamiento de inferencia proporciona flexibilidad. El sistema puede desplegarse de forma conjunta (módulo y servidor en local) en estaciones de trabajo modestas, sin necesidad de GPU, o bien en configuraciones distribuidas, con el servidor de inferencia en una máquina externa capaz de manejar cargas más intensivas. Este enfoque

asegura que la solución pueda adaptarse a distintos contextos, desde pequeños laboratorios con recursos limitados hasta entornos forenses de mayor escala.

Desde el punto de vista del perito, el uso de la herramienta se integra de forma natural en el flujo de trabajo de Autopsy. Una vez cargado un caso y seleccionadas las imágenes a analizar, el analista activa el módulo y elige el modo de trabajo deseado, detección de objetos o búsquedas por similitud, ya sea partir de una imagen o de una consulta textual. El sistema se encarga de enviar las evidencias al servidor de inferencia, que procesa las peticiones y devuelve las predicciones de manera automática. Estos resultados se registran como artefactos vinculados al caso, incluyendo las etiquetas generadas, el modelo responsable y el nivel de confianza alcanzado. Gracias a esta integración, el investigador puede filtrar, priorizar y recorrer las imágenes asociadas a cada categoría con apenas unos clics, lo que simplifica de forma notable la revisión de grandes volúmenes de material. Además, permite integrarse con las funcionalidades nativas de Autopsy, como por ejemplo, incluir los artefactos generados por nuestro módulo en un informe HTML o PDF.

Un aspecto esencial del diseño es la preservación de la trazabilidad de extremo a extremo. Cada resultado generado por los modelos queda inequívocamente asociado a su imagen de origen (véase Sección 5.2.4), acompañado de metadatos que incluyen el identificador del archivo, el modelo que produjo la predicción y el nivel de confianza. Estos elementos permiten no solo justificar el hallazgo en un informe técnico, sino también repetir el análisis bajo las mismas condiciones, garantizando así la transparencia y reproducibilidad exigidas en un contexto judicial. Esta trazabilidad asegura que el sistema no se limite a ofrecer sugerencias automáticas, sino que cada inferencia pueda ser auditada y defendida como parte del procedimiento forense.

En relación con los objetivos definidos en la Sección 1.2, los resultados alcanzados permiten afirmar que todos ellos han sido cumplidos de manera satisfactoria. Este cumplimiento no se limita únicamente a la integración funcional en Autopsy, sino que también se ha demostrado mediante la validación de casos prácticos y la evaluación técnica del rendimiento en entornos realistas de laboratorio (véase Capítulo 6). A continuación, se justifica el cumplimiento de cada uno de los objetivos específicos marcados al inicio del proyecto:

- **OBJ1:** *Desarrollar un módulo que permita al investigador forense buscar y clasificar imágenes utilizando varios modelos de IA integrados en el flujo de trabajo de Autopsy.* Este objetivo se ha alcanzado mediante el desarrollo de un módulo plenamente funcional, estable y configurable que se integra de forma nativa en Autopsy y asiste en el flujo de trabajo habitual del analista. Las imágenes se procesan automáticamente y los resultados aparecen registrados como artefactos forenses en la interfaz de Autopsy, manteniendo en todo momento la trazabilidad con la evidencia original. El diseño, implementación y pruebas del módulo se describen en detalle en el Capítulo 5, mientras que su validación

práctica se recoge en la Capítulo 6. Para reforzar su fiabilidad operativa, se incorporaron validaciones en el punto de entrada (rechazando peticiones incompletas o mal formadas), una gestión controlada de errores en la inferencia (con respuestas HTTP claras y comprensibles) y un sistema de *logging* que documenta eventos críticos como la carga de modelos, el cálculo dinámico de lotes o las excepciones detectadas. Gracias a estas medidas, el módulo garantiza continuidad de servicio, diagnósticos rápidos en caso de fallo y seguridad en el manejo de la evidencia.

- **OBJ2:** *Integrar funcionalidades de detección y clasificación de objetos, búsqueda por similitud de imagen y búsqueda por similitud de texto, automatizando el etiquetado y la agrupación de imágenes según su contenido, facilitando así las investigaciones forenses.* La herramienta incorpora capacidades avanzadas de visión por computador que automatizan el análisis visual de la evidencia. Se han integrado modelos de detección de objetos (YOLOv8, YOLOv11, Faster R-CNN y SSD) y de búsqueda por similitud mediante CLIP, tanto en la modalidad de búsqueda de texto a imagen como en la de búsqueda de imagen a imagen. La motivación y criterios de elección de estos modelos se detallan en el Capítulo 3. En las pruebas descritas en la Sección 6.1, el sistema identificó con éxito objetos relevantes para la investigación forense, como personas, dispositivos electrónicos o armas blancas. También localizó imágenes similares dentro de grandes colecciones, resolviendo consultas semánticas complejas, como "mujer con cuchillo", todo ello sin necesidad de etiquetado previo.

- **OBJ3:** *Evaluar la efectividad del módulo mediante casos de uso y métricas forenses (precisión, rendimiento y tiempo de respuesta), asegurando su fiabilidad y utilidad en escenarios reales de investigación.* La evaluación técnica confirmó la viabilidad del sistema en escenarios realistas de laboratorio. Los experimentos, métricas de precisión y tiempos de respuesta se documentan en el Capítulo 6, especialmente en la Seccion 6.2 En equipos sin aceleración por GPU, los modelos ligeros (p. ej., YOLOv8 o CLIP) lograron procesar lotes de imágenes en apenas unos segundos, lo que los convierte en opciones idóneas para cribados rápidos. Modelos más exigentes (p. ej., Faster R-CNN) ofrecieron mayor precisión a costa de un tiempo de ejecución más elevado, lo que proporciona al perito un abanico de alternativas en función de las prioridades de cada caso (rapidez frente a exactitud). En todos los casos, el sistema se comportó de manera estable, eficiente e integrada con Autopsy, demostrando así su aplicabilidad práctica en entornos forenses con recursos limitados.

De forma ilustrativa, la herramienta muestra los resultados directamente en la interfaz de Autopsy, integrándolos como artefactos forenses que el analista puede consultar y organizar con facilidad. Las Figuras 5.5 y 5.6 presentan un ejemplo de la visualización de dichos

artefactos en Autopsy. A su vez, las Figuras 5.7 y 5.9 profundizan en el sistema de etiquetado, mostrando cómo cada imagen puede recibir varias etiquetas simultáneas, y cómo estas categorías se agrupan de forma jerárquica dentro del panel de resultados. Además, cada imagen asociada a un artefacto puede previsualizarse de inmediato, lo que facilita la revisión comparada y el filtrado ágil de evidencias. En cada registro se conserva, además, la información del modelo utilizado y el nivel de confianza obtenido, aportando así transparencia y apoyo directo a la toma de decisiones del investigador.

Aunque los resultados obtenidos demuestran la utilidad práctica de la herramienta, es necesario señalar sus limitaciones. Todas las validaciones se han realizado en equipos sin GPU (véase Sección 4.3), por lo que el rendimiento alcanzado está condicionado por el hardware disponible y el volumen de imágenes a procesar. Además, el ajuste del umbral de confianza en tareas de detección y similitud resulta determinante: valores conservadores reducen la probabilidad de falsos positivos, pero pueden omitir hallazgos marginales; mientras que umbrales más permisivos aumentan la cobertura a costa de requerir mayor revisión manual. En escenarios muy densos o ambiguos, la intervención directa del perito sigue siendo imprescindible, ya que la herramienta no sustituye su criterio experto, sino que lo complementa acelerando la fase inicial de cribado.

En términos operativos, la herramienta adquiere un valor especial en tres situaciones recurrentes en el trabajo forense: (i) el cribado inicial de grandes volúmenes de imágenes en dispositivos incautados, donde permite priorizar rápidamente el contenido que merece un análisis más detallado; (ii) las búsquedas dirigidas por indicios específicos, como la detección de armas blancas o dispositivos electrónicos, que ayudan a confirmar o descartar hipótesis de investigación; y (iii) los análisis comparativos basados en similitud, útiles para identificar patrones o entidades repetidas en múltiples archivos. En todos estos escenarios, la integración directa con Autopsy asegura que la organización de la evidencia y su trazabilidad se mantengan intactas, sin fricciones en el flujo de trabajo del laboratorio.

Además de los objetivos inicialmente planteados, el proyecto ha generado resultados complementarios de notable interés. En particular, la publicación de la herramienta como software libre bajo licencia GNU General Public License v3.0 (GPLv3) [92], disponible en el repositorio público de GitHub *autopsy-ai-image-toolkit* [93], donde se ofrece también el código fuente completo acompañado de la documentación técnica y las instrucciones de instalación y uso (véase Apéndice A). Esta decisión garantiza su accesibilidad y fomenta la colaboración de la comunidad científica y profesional, facilitando que otros investigadores puedan reutilizar, adaptar y mejorar la solución. Asimismo, se elaboró documentación técnica detallada que explica los pasos de instalación, configuración y despliegue en distintos entornos operativos, lo que refuerza su utilidad tanto en escenarios académicos como profesionales. Estos logros, aunque no contemplados en los objetivos iniciales, amplían el alcance del proyecto y consolidan

su valor como contribución práctica al ámbito de la informática forense.

En conjunto, los resultados obtenidos evidencian que se han alcanzado con éxito todos los objetivos específicos definidos al inicio del proyecto, lo que implica también el cumplimiento del objetivo general planteado. La solución desarrollada aporta un valor tangible en el ámbito del análisis forense digital al reducir de manera significativa la carga de trabajo del investigador y automatizar tareas clave sin alterar el flujo operativo habitual de Autopsy. Más que un sustituto del perito, la herramienta se concibe como un asistente inteligente que refuerza su capacidad de análisis, acelera los procesos de cribado y facilita la identificación temprana de evidencias relevantes.

Capítulo 8

Conclusiones y trabajo futuro

Este capítulo recoge las conclusiones finales del trabajo realizado y plantea posibles líneas de evolución futura. Se sintetizan los logros más relevantes alcanzados, se valora el impacto de la herramienta en el ámbito de la informática forense y se señalan direcciones que podrían potenciar su utilidad y alcance en próximos desarrollos.

8.1 Conclusiones

La creciente necesidad de herramientas eficientes en el ámbito de la informática forense ha impulsado el desarrollo de soluciones capaces de hacer frente al volumen y la complejidad de los datos digitales. En particular, el análisis de imágenes representa un desafío crítico, ya que los peritos se enfrentan a grandes cantidades de contenido visual cuya inspección manual no escala adecuadamente. Este proyecto surge como respuesta a esa necesidad, proponiendo un módulo para Autopsy que integra técnicas de IA orientadas al análisis automatizado de imágenes.

A lo largo del trabajo se ha diseñado e implementado un sistema funcional compuesto por dos bloques principales: un módulo para Autopsy que gestiona las imágenes de los casos forenses y un servidor externo en FastAPI que ejecuta modelos de visión por computador. Esta arquitectura cliente-servidor ofrece flexibilidad de despliegue, desde análisis locales hasta entornos distribuidos, y proporciona al perito funcionalidades avanzadas como detección de objetos y búsquedas por similitud, tanto semántica (texto) como visual (imagen), accesibles directamente desde la interfaz de Autopsy.

Las pruebas realizadas muestran que el sistema reduce de forma significativa la carga de trabajo del analista, identificando con rapidez y precisión las imágenes más relevantes en cada caso. Los modelos integrados han demostrado un equilibrio adecuado entre velocidad y exactitud, lo que permite adaptar su uso a distintos escenarios forenses. Además, el módulo ha sido validado en condiciones de uso realistas, confirmando su viabilidad como herramienta

de apoyo en investigaciones criminales.

A nivel académico, este proyecto ha permitido consolidar conocimientos adquiridos durante el grado, especialmente en áreas como visión por computador, desarrollo de aplicaciones web con FastAPI y programación en entornos forenses como Autopsy. También ha sido clave para comprender los retos técnicos y legales asociados al tratamiento automatizado de evidencias digitales, lo que refuerza la importancia de desarrollar soluciones transparentes, auditables y respetuosas con los principios del análisis forense.

En el plano personal, esta experiencia ha supuesto una oportunidad única para aplicar conocimientos teóricos en un entorno práctico real. La construcción de una herramienta que puede tener impacto directo en investigaciones judiciales ha reforzado mi interés en la intersección entre IA e informática forense, un campo que combina desafíos técnicos con una fuerte dimensión social. En el futuro, me gustaría seguir profundizando en este ámbito, ampliando mis competencias tanto a nivel técnico como en materia de protección de datos y regulación.

8.2 Trabajo futuro

Aunque el sistema desarrollado ha cumplido con los objetivos principales del proyecto, durante el proceso de validación se han identificado limitaciones y oportunidades de mejora que abren la puerta a ampliar su alcance y adaptabilidad. Entre las líneas más prometedoras destacan:

- **Integración de nuevos modelos de IA:** incorporar modelos de última generación como RT-DETR o Segment Anything Model (SAM) [94], que permiten una detección más precisa y segmentación semántica avanzada sobre imágenes complejas.

- **Análisis forense multimodal:** ampliar el módulo para procesar otros tipos de contenido más allá de imágenes, como vídeos, documentos o audios, permitiendo correlacionar información visual con texto extraído mediante OCR o datos de metadatos.

- **Conexión con bases de datos externas:** permitir que el sistema compare automáticamente imágenes del caso con contenidos visuales alojados en bases de datos forenses, como listados de objetos robados o personas desaparecidas, facilitando la identificación rápida de coincidencias relevantes.

- **Integración de asistentes de IA y agentes autónomos:** explorar la incorporación de modelos de lenguaje (LLM) [95, 96] en Autopsy, ya sea mediante un chat integrado en la interfaz o a través de servidores MCP [97], para asistir al perito en la consulta y análisis de evidencias.

Estas líneas de trabajo apuntan hacia un sistema forense más inteligente, modular y centrado en el usuario, capaz de integrar nuevas modalidades de análisis y de responder a las demandas cambiantes del ecosistema digital. Su evolución futura puede contribuir a consolidar herramientas abiertas, transparentes y de gran impacto en la práctica forense.

Apéndices

Apéndice A

Manual de instalación y despliegue

Este apéndice describe el proceso de instalación del módulo de Autopsy desarrollado en este proyecto y el despliegue del servidor de inferencia que le da soporte. Su objetivo es servir como manual de usuario, de modo que cualquier persona pueda reproducir el sistema completo.

A.1 Instalación del módulo en Autopsy

El módulo dota a Autopsy de capacidades de visión por computador, enviando las imágenes del caso a un servidor de inferencia que las procesa y devuelve los resultados directamente en la interfaz para su posterior análisis. La instalación del módulo requiere cumplir una serie de requisitos de software y seguir una secuencia ordenada de pasos para que quede plenamente integrado en la aplicación.

Requisitos previos

Para que el módulo funcione correctamente se necesita:

- Windows 10 (probado en versión 22H2).
- Autopsy 4.19 con soporte para módulos Python vía Jython.
- Jython 2.7.4.
- Java OpenJDK 17 (probado con Temurin-17.0.14).
- Python 3.11.5 (solo para tareas auxiliares, no para la ejecución principal).

Procedimiento de instalación

1. Descarga del código. El primer paso consiste en obtener el código fuente desde el repositorio oficial de GitHub, bien clonando el repositorio o descargando el paquete comprimido:

```
git clone https://github.com/manuelnoyav/autopsy-ai-image-toolkit.git
```

2. Copia en el directorio de módulos de Autopsy. Una vez descargado, se debe copiar la carpeta *AI_Image_Module* en la ruta de módulos de Autopsy, de forma que quede accesible como complemento:

```
C:\Users\<usuario>\AppData\Roaming\Autopsy\python_modules\
```

3. Configuración del servidor de inferencia. El archivo *AI_Image_Module.py* contiene el parámetro *server_url*, que debe señalar la dirección del servidor de IA. Por ejemplo, para un despliegue local:

```
self.server_url = "http://localhost:8000/process"
```

Si se desea cambiar la ubicación del archivo de registro del módulo, puede ajustarse la constante *LOG_FILE_PATH* (aprox. línea 39) indicando una ruta de preferencia.

4. Verificación y prueba. Finalmente, se debe iniciar Autopsy, crear o abrir un caso de prueba y verificar que el nuevo módulo aparece en la sección *ingest modules*. Se recomienda ejecutar una ingestión de imágenes de ejemplo para confirmar que la comunicación con el servidor de IA funciona correctamente y que los resultados se muestran en la interfaz. En caso de fallo, conviene revisar los registros generados en la ruta definida por *LOG_FILE_PATH* para identificar posibles incidencias de configuración.

A.2 Despliegue del servidor de IA

El servidor expone una API REST que ejecuta los modelos de visión por computador (YOLOv8/v11, SSD, Faster R-CNN y CLIP) a los que se conecta el módulo de Autopsy. La correcta puesta en marcha del servidor garantiza que el módulo de Autopsy pueda enviar las imágenes y recibir las inferencias sin interrupciones.

Requisitos previos

- Docker 20.10 o superior.
- Carpeta *models* con los pesos de los modelos necesarios (por ejemplo *yolov8n.pt* y *yolo11n.pt*).

Construcción y ejecución

Situarse en el directorio del proyecto del servidor y construir la imagen Docker:

```
docker build -t ai-server-cpu:latest .
```

A continuación, ejecutar el contenedor en segundo plano, exponiendo el puerto 8000 y montando la carpeta de modelos:

```
docker run -d ^
  --name ai-server-cpu ^
  -p 8000:8000 ^
  -v "%cd%\models:/app/models" ^
  ai-server-cpu:latest
```

En sistemas Linux o macOS, sustituir las continuaciones `^` por `\` y `%cd%` por `$(pwd)`.

Comprobación y gestión

Para confirmar que el servidor arranca sin problemas:

```
docker logs -f ai-server-cpu
```

Algunos comandos útiles para su administración son:

```
docker stop ai-server-cpu     # detiene el contenedor
docker start ai-server-cpu    # inicia el contenedor
docker restart ai-server-cpu  # reinicia el contenedor
docker rm ai-server-cpu       # elimina el contenedor
```

Uso de la API

El punto de entrada principal de la API es:

```
POST /process
```

Este endpoint recibe imágenes codificadas en base64 junto con el modelo seleccionado y devuelve los resultados en formato JSON. Antes de integrar el servidor con Autopsy se recomienda comprobar manualmente el funcionamiento del endpoint, por ejemplo mediante *curl* o una herramienta como Postman, para asegurarse de que responde correctamente.

Con estos pasos, cualquier usuario puede instalar el módulo en Autopsy, desplegar el servidor de IA y disponer de un sistema completo para detección de objetos, búsqueda por similitud y consultas semánticas, plenamente integrado en el flujo de trabajo forense.

Bibliografía

[1] Europol, "Internet organised crime threat assessment (iocta) 2023," Luxembourg, 2023. [En línea]. Disponible en: https://data.europa.eu/doi/10.2813/587536

[2] INCIBE, "Balance de ciberseguridad 2024," 2025, informe técnico en formato infografía. [En línea]. Disponible en: https://www.incibe.es/sites/default/files/Comunicaci%C3%B3n_2025/Infograf%C3%ADa_BalanceCiberseguridad_INCIBE_2024_web.pdf

[3] International Organization for Standardization, "ISO/IEC 27037:2012 — Directrices para la identificación, recolección, adquisición y preservación de evidencia digital," https://www.iso.org/standard/44381.html, 2012, accedido el 11 septiembre 2025.

[4] España, "Ley 1/2000, de 7 de enero, de Enjuiciamiento Civil," https://www.boe.es/buscar/act.php?id=BOE-A-2000-323, 2000, artículos 335 y ss. Definen la figura del perito como experto que aporta conocimientos científicos, técnicos o prácticos para valorar hechos relevantes en el proceso. Accedido el 11 septiembre 2025.

[5] ——, "Ley de Enjuiciamiento Criminal," https://www.boe.es/buscar/act.php?id=BOE-A-1882-6036, 1882, artículos 456–485. Regulan la actuación de los peritos en causas penales. Accedido el 11 septiembre 2025.

[6] A. M. Shimaoka, R. C. Ferreira, and A. Goldman, "The evolution of CRISP-DM for Data Science: Methods, Processes and Frameworks," *SBC Reviews on Computer Science*, vol. 4, no. 1, pp. 28–43, Oct. 2024, number: 1. [En línea]. Disponible en: https://journals-sol.sbc.org.br/index.php/reviews/article/view/3757

[7] Europol, "SIRIUS EU Digital Evidence Situation Report 2021," https://www.europol.europa.eu/publications-events/publications/sirius-eu-digital-evidence-situation-report-2021, 2021, "The growth of volume of available personal data was accelerated by intense use of social media and the proliferation of connected devices." Accedido el 11 septiembre 2025.

[8] National Institute of Standards and Technology, "Digital forensics: NIST guidance and resources," https://www.nist.gov/forensics/digital-forensics, s.f., "Digital forensic investigators face challenges such as … locating individual items of evidence among vast quantities of data." Accedido el 11 septiembre 2025.

[9] European Union Agency for Cybersecurity., *ENISA threat landscape 2024: July 2023 to June 2024.* LU: Publications Office, 2024. [En línea]. Disponible en: https://data.europa.eu/doi/10.2824/0710888

[10] C. M. Miller, "A survey of prosecutors and investigators using digital evidence: A starting point," *Forensic Science International: Synergy*, vol. 6, p. 100296, Jan. 2023. [En línea]. Disponible en: https://www.sciencedirect.com/science/article/pii/S2589871X2200081X

[11] D. Dunsin, M. C. Ghanem, K. Ouazzane, and V. Vassilev, "A Comprehensive Analysis of the Role of Artificial Intelligence and Machine Learning in Modern Digital Forensics and Incident Response," *Forensic Science International: Digital Investigation*, vol. 48, p. 301675, Mar. 2024, arXiv:2309.07064 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2309.07064

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," May 2016, arXiv:1506.02640 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1506.02640

[13] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," Jan. 2016, arXiv:1506.01497 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1506.01497

[14] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," 2016, vol. 9905, pp. 21–37, arXiv:1512.02325 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1512.02325

[15] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," Feb. 2021, arXiv:2103.00020 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2103.00020

[16] F. Focus, "Griffeye Brain In Analyze DI Pro," Nov. 2019. [En línea]. Disponible en: https://www.forensicfocus.com/reviews/griffeye-brain-in-analyze-di-pro/

[17] S. L. Sanna, L. Regano, D. Maiorca, and G. Giacinto, "Exploring the Robustness of AI-Driven Tools in Digital Forensics: A Preliminary Study," Dec. 2024, arXiv:2412.01363 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2412.01363

[18] "Autopsy User Documentation: Autopsy User's Guide." [En línea]. Disponible en: https://sleuthkit.org/autopsy/docs/user-docs/4.22.0//

[19] C. Zheng, A. Shrivastava, and A. Owens, "EXIF as Language: Learning Cross-Modal Associations Between Images and Camera Metadata," Jun. 2023, arXiv:2301.04647 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2301.04647

[20] X. Du, N.-A. Le-Khac, and M. Scanlon, "Evaluation of Digital Forensic Process Models with Respect to Digital Forensics as a Service," Aug. 2017, arXiv:1708.01730 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1708.01730

[21] "Snorting video appearing to show former SA opposition leader a 'complete shock' to Liberal successor," *ABC News*, Sep. 2024. [En línea]. Disponible en: https://www.abc.net.au/news/2024-09-10/sa-liberal-former-leader-david-speirs-snorting-video/104330642

[22] F. Casino, T. Dasaklis, G. Spathoulas, M. Anagnostopoulos, A. Ghosal, I. Borocz, A. Solanas, M. Conti, and C. Patsakis, "Research trends, challenges, and emerging topics of digital forensics: A review of reviews," Jan. 2022, arXiv:2108.04634 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2108.04634

[23] Department of Computing Sciences and Engineering, Vellore Institute of Technology, Bhopal (M.P), India., D. Banwani, Y. Kalra, and Department of Computing Sciences and Engineering, Vellore Institute of Technology, Bhopal (M.P), India., "Maintaining and Evaluating the Integrity of Digital Evidence in Chain of Custody," *International Journal of Recent Technology and Engineering (IJRTE)*, vol. 10, no. 3, pp. 90–96, Sep. 2021. [En línea]. Disponible en: https://www.ijrte.org/wp-content/uploads/papers/v10i3/C64490910321.pdf

[24] D. Brezinski and T. Killalea, "Guidelines for evidence collection and archiving," RFC 3227, Internet Engineering Task Force (IETF), Tech. Rep., Feb. 2002, request for Comments 3227. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc3227.html

[25] A. Patel and W. A. P. Smith, "Manifold-based constraints for operations in face space," *Pattern Recognition*, vol. 52, pp. 206–217, Apr. 2016. [En línea]. Disponible en: https://www.sciencedirect.com/science/article/pii/S0031320315003702

[26] L. Giovannangeli, R. Bourqui, R. Giot, and D. Auber, "Toward automatic comparison of visualization techniques: Application to graph visualization," *Visual Informatics*, vol. 4, no. 2, pp. 86–98, Jun. 2020. [En línea]. Disponible en: https://www.sciencedirect.com/science/article/pii/S2468502X20300140

[27] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," May 2020, arXiv:2005.12872 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2005.12872

[28] "Introducing Gemini: our largest and most capable AI model," Dec. 2023. [En línea]. Disponible en: https://blog.google/technology/ai/google-gemini-ai/

[29] Z. Yang, L. Li, K. Lin, J. Wang, C.-C. Lin, Z. Liu, and L. Wang, "The Dawn of LMMs: Preliminary Explorations with GPT-4V(ision)," Oct. 2023, arXiv:2309.17421 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2309.17421

[30] "FTK Forensics Toolkit - Digital Forensics Software Tools." [En línea]. Disponible en: https://www.exterro.com/digital-forensics-software/forensic-toolkit

[31] "Digital Forensics Software | OpenText Forensic." [En línea]. Disponible en: https://www.opentext.com/es-es/productos/forensic

[32] "X-Ways Forensics: Integrated Computer Forensics Software." [En línea]. Disponible en: https://www.x-ways.net/forensics/

[33] F. Focus, "BlackLight From BlackBag," Feb. 2017. [En línea]. Disponible en: https://www.forensicfocus.com/reviews/blacklight-from-blackbag/

[34] Signiant, "Kyno | Media Management and Workflow Software." [En línea]. Disponible en: https://lesspain.software/kyno/

[35] "CAINE Live USB/DVD - computer forensics digital forensics." [En línea]. Disponible en: https://www.caine-live.net/

[36] "The Sleuth Kit." [En línea]. Disponible en: https://www.sleuthkit.org/sleuthkit/

[37] "Autopsy User Documentation: Image Gallery Module." [En línea]. Disponible en: https://sleuthkit.org/autopsy/docs/user-docs/4.5.0/image_gallery_page.html

[38] "Autopsy User Documentation: Hash Lookup Module." [En línea]. Disponible en: https://sleuthkit.org/autopsy/docs/user-docs/4.8.0/hash_db_page.html

[39] N. Islam, Z. Islam, and N. Noor, "A Survey on Optical Character Recognition System," Oct. 2017, arXiv:1710.05703 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1710.05703

[40] "tesseract-ocr/tesseract," Jul. 2025, original-date: 2014-08-12T18:04:59Z. [En línea]. Disponible en: https://github.com/tesseract-ocr/tesseract

[41] "eDiscovery with Belkasoft." [En línea]. Disponible en: https://belkasoft.com/ediscovery_with_belkasoft

[42] "MOBILedit Forensic details." [En línea]. Disponible en: https://www.mobiledit.com/mobiledit-forensic/details

[43] M. Steinebach, "An Analysis of PhotoDNA," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, ser. ARES '23. New York, NY, USA: Association for Computing Machinery, Aug. 2023, pp. 1–8. [En línea]. Disponible en: https://dl.acm.org/doi/10.1145/3600160.3605048

[44] "Project VIC." [En línea]. Disponible en: https://www.projectvic.org/project-vic

[45] J. Gao, Q. Jiang, B. Zhou, D. Chen, J. Gao, Q. Jiang, B. Zhou, and D. Chen, "Convolutional neural networks for computer-aided detection or diagnosis in medical image analysis: An overview," *Mathematical Biosciences and Engineering*, vol. 16, no. 6, pp. 6536–6561, 2019. [En línea]. Disponible en: http://www.aimspress.com/article/doi/10.3934/mbe.2019326

[46] J. Dai, Y. Li, K. He, and J. Sun, "R-FCN: Object Detection via Region-based Fully Convolutional Networks," Dec. 2023, arXiv:1605.06409 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1605.06409

[47] Ultralytics, "RT-DETR (Transformador de detección en tiempo real)." [En línea]. Disponible en: https://docs.ultralytics.com/es/models/rtdetr

[48] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," Jun. 2021, arXiv:2010.11929 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2010.11929

[49] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common Objects in Context," Feb. 2015, arXiv:1405.0312 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1405.0312

[50] "Modelo de desarrollo incremental iterativo de software." [En línea]. Disponible en: https://mwebs.com.uy/blog/qu%C3%A9-es-el-desarrollo-incremental/23

[51] M. Poppendieck and T. Poppendieck, *Lean Software Development: An Agile Toolkit*. Addison-Wesley Professional, 2003. [En línea]. Disponible en: https://www.oreilly.com/library/view/lean-software-development/0321150783/

[52] Atlassian, "¿Qué es un tablero kanban?" [En línea]. Disponible en: https://www.atlassian.com/es/agile/kanban/boards

[53] "Git version control: A comprehensive guide to modern software development," vol. 8. [En línea]. Disponible en: https://iaeme.com/MasterAdmin/Journal_uploads/IJRCAIT/VOLUME_8_ISSUE_1/IJRCAIT_08_01_043.pdf

[54] J. Chen, "Model Algorithm Research based on Python Fast API," *Frontiers in Science and Engineering*, vol. 3, no. 9, pp. 7–10, Sep. 2023, number: 9. [En línea]. Disponible en: https://bcpublication.org/index.php/FSE/article/view/5591

[55] "Memoria de modificación del plan de estudios del grado en ingeniería informática," [En línea], 2017, disponible en: https://www.fic.udc.es/sites/default/files/documentos/gei_memoria_modificacion_2017.pdf.

[56] "¿Qué es un diagrama de Gantt y para qué sirve?" Feb. 2025. [En línea]. Disponible en: https://www.obsbusiness.school/blog/que-es-un-diagrama-de-gantt-y-para-que-sirve

[57] "Welcome to Python.org," Sep. 2025. [En línea]. Disponible en: https://www.python.org/

[58] "Jython: Python for the java platform," accessed: 2025-07-04. [En línea]. Disponible en: https://www.jython.org/

[59] "Visual Studio Code - Code Editing. Redefined." [En línea]. Disponible en: https://code.visualstudio.com/

[60] GitHub. (2025) Github codespaces. [En línea; consultado el 5 de septiembre de 2025]. [En línea]. Disponible en: https://github.com/features/codespaces

[61] Ultralytics, "Yolo11: Official documentation," https://docs.ultralytics.com/models/yolo11/, 2025.

[62] Ultralytics, "Yolov8: Official documentation," https://docs.ultralytics.com/es/models/yolov8/, 2025.

[63] PyTorch, "Faster r-cnn — torchvision main documentation," https://docs.pytorch.org/vision/main/models/faster_rcnn.html, 2025.

[64] PyTorch, "Ssd300_vgg16 — torchvision main documentation," https://docs.pytorch.org/vision/main/models/generated/torchvision.models.detection.ssd300_vgg16.html, 2025.

[65] OpenAI, "Clip: Connecting text and images," https://openai.com/research/clip, jan 2021.

[66] "Pillow." [En línea]. Disponible en: https://pillow.readthedocs.io/en/stable/index.html

[67] G. Rodola. (2025) psutil: Cross-platform library for process and system monitoring in python. Accedido: 5 de septiembre de 2025. [En línea]. Disponible en: https://github.com/giampaolo/psutil

[68] D. Merkel, "Docker: lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, p. 2:2, Mar. 2014.

[69] P. P. Kore, M. J. Lohar, M. T. Surve, and S. Jadhav, "API Testing Using Postman Tool," *International Journal for Research in Applied Science and Engineering Technology*, vol. 10, no. 12, pp. 841–843, Dec. 2022. [En línea]. Disponible en: https://www.ijraset.com/best-journal/api-testing-using-postman-tool

[70] Universidade da Coruña, "Calculadora de contratos de la universidade da coruña," https://artesa.udc.gal/calculadoraContratos/, 2025, [En línea; consultado el 4-jul-2025].

[71] Centro de Investigación en Tecnoloxías da Información e as Comunicacións (CITIC), "Sitio web oficial del citic," https://citic.udc.es/, 2025, [En línea; consultado el 4-jul-2025].

[72] C. Cloud, "What is the Latest Pricing for 32+ vCPU, 128GB+ RAM, High Storage?" [En línea]. Disponible en: https://cyfuture.cloud/kb/storage/what-is-the-latest-pricing-for-32-vcpu-128gb-ram-high-storage

[73] R. T. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content," Internet Engineering Task Force, Request for Comments RFC 7231, Jun. 2014, num Pages: 101. [En línea]. Disponible en: https://datatracker.ietf.org/doc/rfc7231

[74] H. Zhang, "Architecture of Network and Client-Server model," Jul. 2013, arXiv:1307.6665 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1307.6665

[75] "Autopsy: Python Tutorial #2: Writing a Data Source Ingest Module." [En línea]. Disponible en: https://sleuthkit.org/autopsy/docs/api-docs/4.20.0/mod_python_ds_ingest_tutorial_page.html

[76] "The Java® Virtual Machine Specification." [En línea]. Disponible en: https://docs.oracle.com/javase/specs/jvms/se8/html/

[77] "javax.swing (Java Platform SE 8)." [En línea]. Disponible en: https://docs.oracle.com/javase/8/docs/api/index.html?javax/swing/package-summary.html

[78] Y. N. Silva, I. Almeida, and M. Queiroz, "SQL: From Traditional Databases to Big Data," in *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*, ser. SIGCSE '16. New York, NY, USA: Association for Computing Machinery, Feb. 2016, pp. 413–418. [En línea]. Disponible en: https://doi.org/10.1145/2839509.2844560

[79] "Database v7.2 Schema - SleuthKitWiki." [En línea]. Disponible en: https://wiki.sleuthkit.org/index.php?title=Database_v7.2_Schema

[80] W. Muła and D. Lemire, "Base64 encoding and decoding at almost the speed of a memory copy," *Software: Practice and Experience*, vol. 50, no. 2, pp. 89–97, Feb. 2020, arXiv:1910.05109 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/1910.05109

[81] "Json," accedido el 7 de septiembre de 2025. [En línea]. Disponible en: https://www.json.org/json-en.html

[82] "Pytorch," PyTorch Foundation, accedido el 7 de septiembre de 2025. [En línea]. Disponible en: https://pytorch.org/

[83] "torchvision — Torchvision 0.22 documentation." [En línea]. Disponible en: https://docs.pytorch.org/vision/stable/index.html

[84] "Microservicios y API REST ¿Cómo se Relacionan? - DEV Community." [En línea]. Disponible en: https://dev.to/marmariadev/microservicios-y-api-rest-como-se-relacionan-5in

[85] "CUDA C++ Programming Guide — CUDA C++ Programming Guide." [En línea]. Disponible en: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[86] "¿Qué significa "sin memoria" (OOM)?" [En línea]. Disponible en: https://phoenixnap.mx/glosario/sin-memoria

[87] Unsplash, "Hermosas imágenes y fotos gratuitas | Unsplash." [En línea]. Disponible en: https://unsplash.com/es

[88] S. Y. Gadre and et al., "DataComp: In search of the next generation of multimodal datasets," Oct. 2023, arXiv:2304.14108 [cs]. [En línea]. Disponible en: http://arxiv.org/abs/2304.14108

[89] "Text-to-Image and Image-to-Image Search Using CLIP | Pinecone." [En línea]. Disponible en: https://www.pinecone.io/learn/clip-image-search/

[90] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The pascal visual object classes (voc) challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[91] F. Roth, "The Inefficiency in Digital Forensics," Jun. 2025. [En línea]. Disponible en: https://cyb3rops.medium.com/the-inefficiency-in-digital-forensics-3da88123d2ec

[92] "The GNU General Public License v3.0 - GNU Project - Free Software Foundation." [En línea]. Disponible en: https://www.gnu.org/licenses/gpl-3.0.en.html#license-text

[93] M. N. Vázquez, "manuelnoyav/autopsy-ai-image-toolkit," Jul. 2025, original-date: 2025-05-07T16:23:30Z. [En línea]. Disponible en: https://github.com/manuelnoyav/autopsy-ai-image-toolkit

[94] "Segment Anything | Meta AI." [En línea]. Disponible en: https://segment-anything.com/

[95] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[96] R. Bommasani *et al.*, "On the opportunities and risks of foundation models," Stanford CRFM, Tech. Rep., 2021.

[97] OpenAI, "Model context protocol specification," https://github.com/modelcontextprotocol/specification, 2024.